

Модель OSI

Отправляющая сторона (инкапсуляция)

Получающая сторона (декапсуляция)

7. Приложения
6. Представления
5. Сеансовый
4. Транспортный
3. Сетевой
2. Канальный
1. Физический

7. Приложения
6. Представления
5. Сеансовый
4. Транспортный
3. Сетевой
2. Канальный
1. Физический

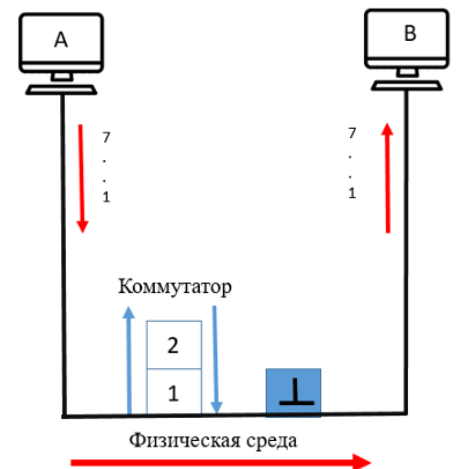
генерация на сетевом, TCP, UDP, шлюзы, firewalls
адресация, маршрутизация, IPv4-6
носим, коммутирование, MAC-адрес, Ethernet
генерация 01001... на физическом

↑
кабели, провода

Инкапсуляция (encapsulation) – помещение данных протоколов более высокого уровня в структуры данных более низкого уровня

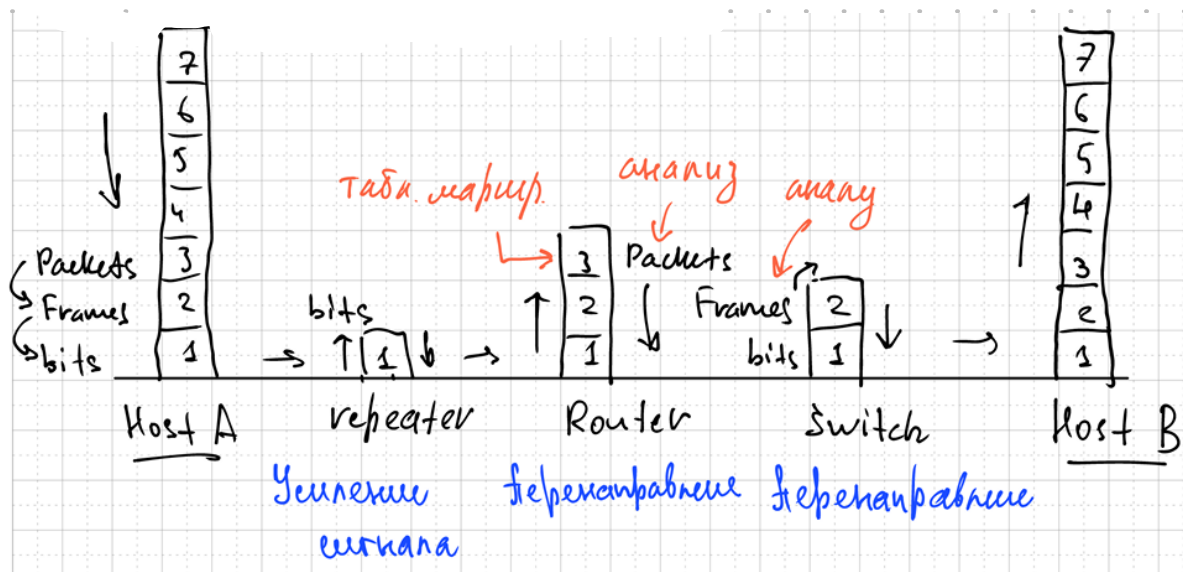
Коммутатор:

- принимает пакеты, 0 и 1
- выполняет деинкапсуляцию (данные идут на 2 уровень ↑)
- обрабатывает данные на 2-м уровне
- выполняет инкапсуляцию (данные идут ↓)



Маршрутизатор:

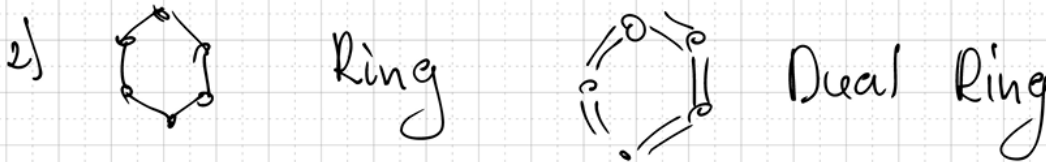
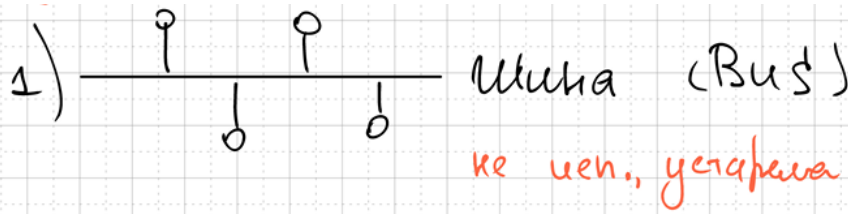
- по всей сети, но деинкапсуляция до 3 уровня



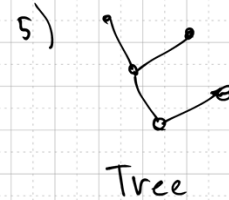
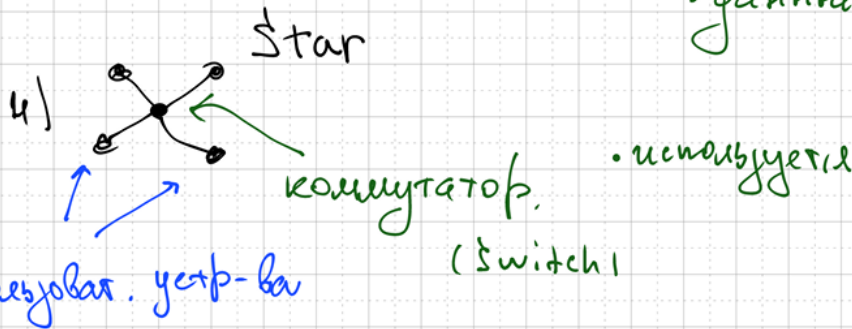
Топология → физ.
→ лог.

Физическая топология — каким образом физически (с помощью кабеля) устройства соединены друг с другом.

Логическая топология — определяет пути передачи данных в сети.

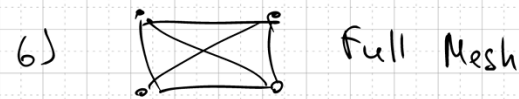
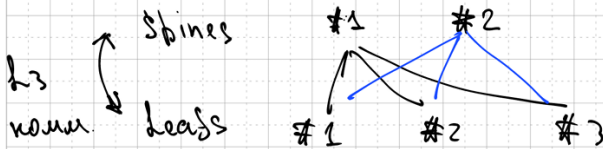


• данные могут идти в обе ст.



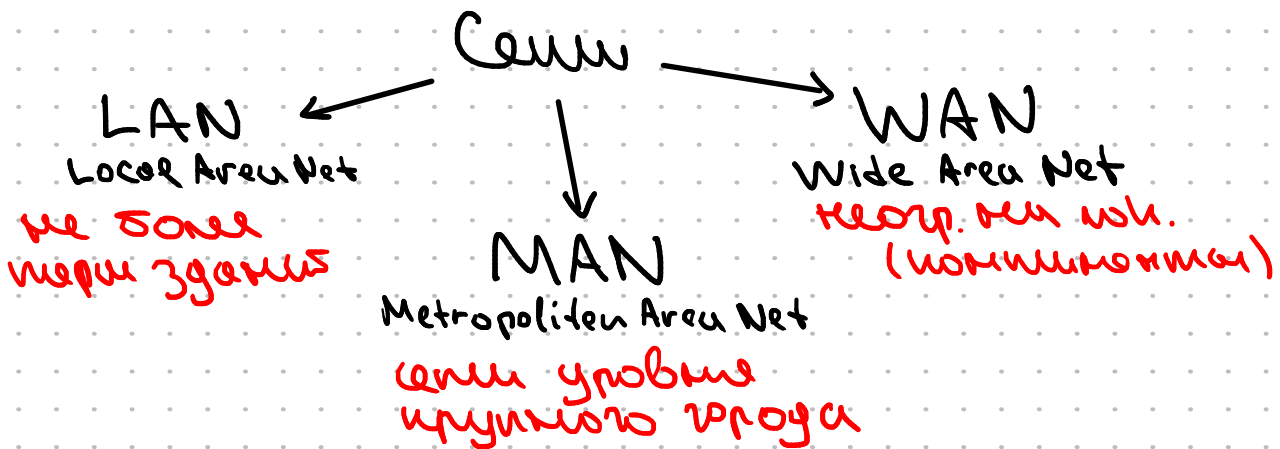
• DC (Clos)

• мен. в диаметре.
• от любого сервера до другого фикс. число переходов: 2

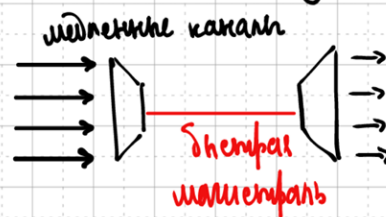


7) Hybrid

ECMP - Equal Cost Multiple Path.



Мультиплексирование/демультиплексирование — это технологии, которые позволяют представить несколько медленных каналов через один быстрый.



• **TDM** - time division multiplexing

- в пакеты записывается время для каждого канала
- выделяются отдельные тайм-слоты для каналов.

• **WDM** - wavelength division multiplexing

- выделяются отдельные длины волн под каждый медленный сигнал.

Среды передачи Ethernet

1) Коаксиальный кабель

• Физ. название — кабель

наимс. кабель

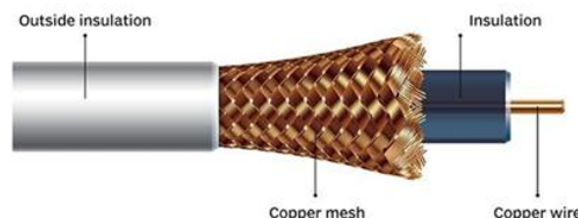


Рисунок 11. Структура коаксиального кабеля.

Диаметр:

200 мм

Путь:

в кабеле в каб. Т-образ. разветвл. с разветвлением

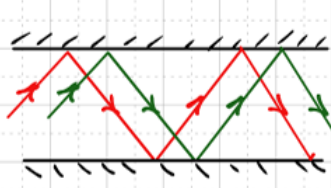
наимс. кабель

500 мм

изменяется прозв. до 40 см. или 1 сек. со скоростью

2) Оптоволоконно

MMF - multimode



50-62,5 мкм

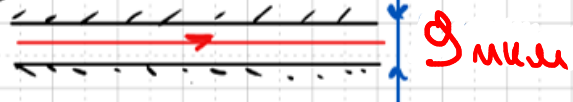
больш.

- передача света по нескольким путям (модам)

≈ 400 м

светодиод

SMF - singlemode



~10 км - макс. дальность

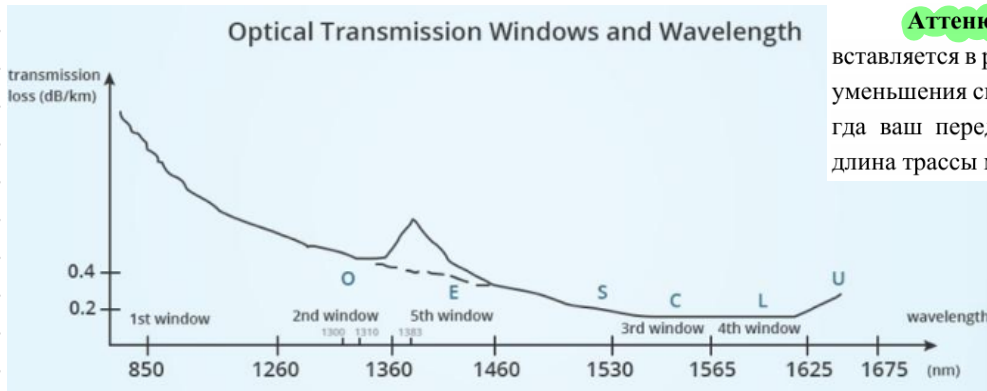
9 мкм

≈ 10 мкм

лазер

Окно прозрачности²⁸ – характеристика волокна.

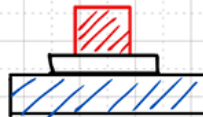
Свет с разной длиной волны затухает по-разному и существует несколько окон прозрачности, где затухание наименьшее. Поэтому сигнал передаётся с такой длиной волны, чтобы попасть в это окно прозрачности.



Аттенюатор (см. Рисунок 32) – устройство, которое вставляется в разрыв оптического кабеля и используется для уменьшения сигнала, чтобы не сжечь приёмник сигнала, когда ваш передатчик и приёмник находятся близко (если длина трассы маленькая, а мощность передатчика высокая).

Самые простые:

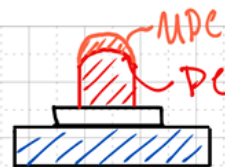
1) Flat – просто соединить



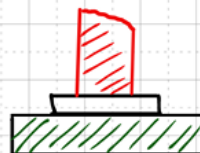
2) PC – Ultra Physical Contact

PC – UPC – с закруглением

• более совершенный тип оптических жил



3) APC – Angled Physical Contact



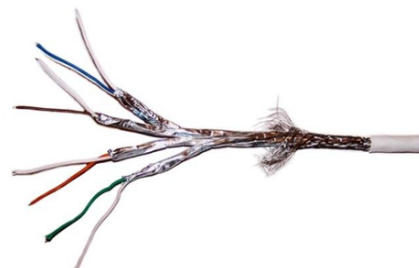
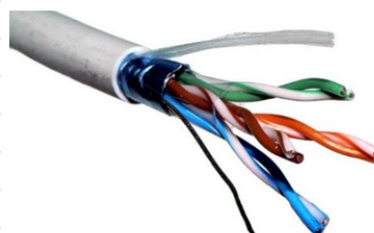
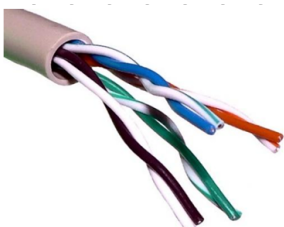
3) Ветви кабеля – выбор между медью и оптоволоком

медь
опт. каб.
2 жилы
U/UTP
Unshielded...

медь
опт. каб.
2 жилы
F/UTP

Foiled Unshielded...

медь
опт. каб.
2 жилы
SF/UTP
Shielded... – better than one shield



А также:

S/FTP

F/FTP

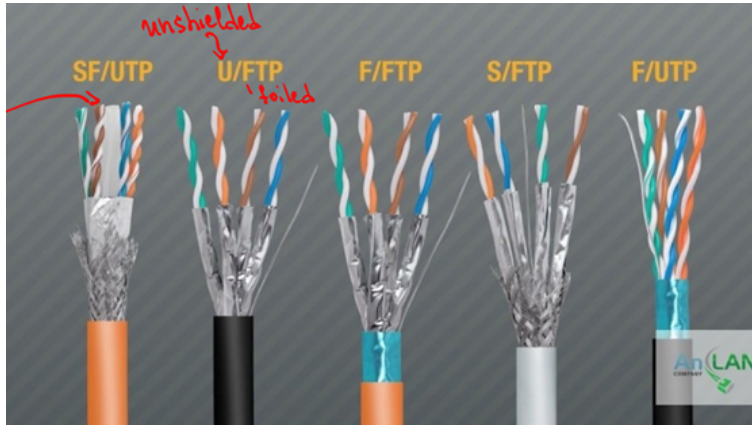
U/FTP

одн. экр.
с медной
оплёткой

одн.
экран

без
одн. экр.

близки к нулю
в экранировании



S - экр. медной
оплёткой

F - экр. оплёткой

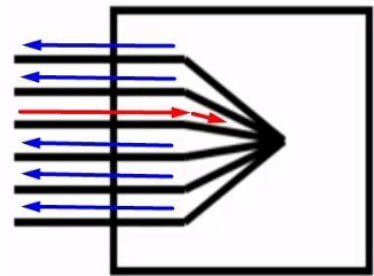
U - без одн. экр.

SF - 2 типа экр.

В нашей сети есть «звезда»,
в центреhub.

Hub (концентратор)

— многопортовый повторитель: устройство, которое
получает битовый поток через один интерфейс и передаёт
(транслирует) через все остальные интерфейсы (см. Рисунок



Стандарты Ethernet

- 10 Мб/с	2 пары		с середины 1980
- 100 Мб/с	2	Fast Ethernet	с 1995 года
- 1 Гб/с	4	Gigabit Ethernet	с 1999 года
- 10 Гб/с	4	10 Gigabit Ethernet	с 2002 года
- 40/100 Гб/с (1 пара)		40/100 Gigabit Ethernet	с 2010 года
2 поколение - с 2010 года			
3 поколение - с 2015 года			
4 поколение - с 2018 года			

Коммутаторы выносятся наружу:

8P8C - оптич. физ. интерфейсы коммутаторов

RJ45 - оптич. коммутатор и распределитель
проводников в нем
(перекрест проводов)

Расширение проводов



На вынос кабеля физ. длина < 100 м.

Длина (123578 м) - максимальная длина

Кабель с **AWG** (24 AWG или 26 AWG) -

- максимальная мощность проводимости
изм. в кабеле проводимости

• **Twisted Pair**

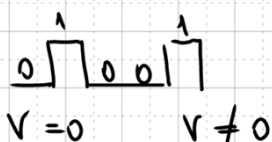
Внутри 2 коакс. кабеля (в дата центрах, дает до 25 Гбит/с),

расстояние 5-10 метров.

AMT MDI/MDIX - не нужно подбирать straight/crossover расположение контактов

PCS Physical Coding Sublayer

1. NRZ (non return to zero)



2. RZ



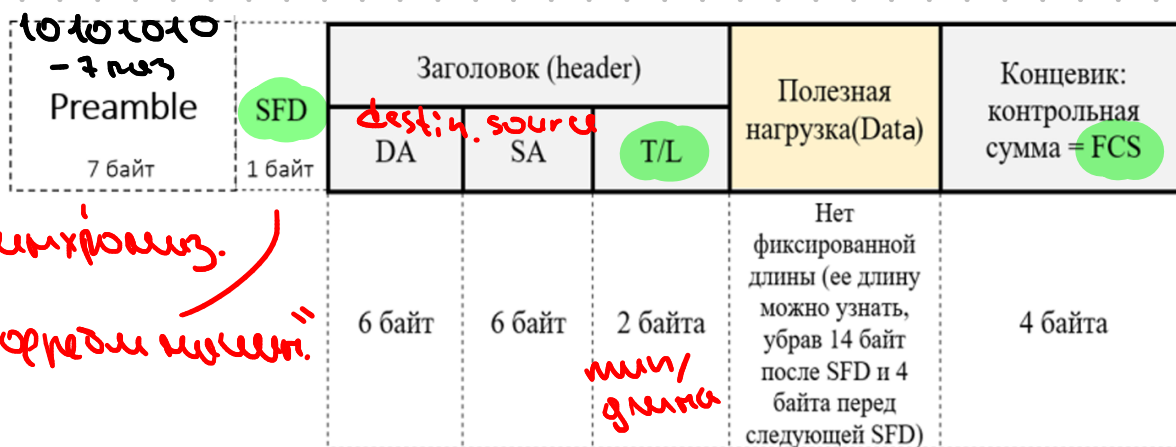
3. Manchester

0: 0 → 1

1: 1 → 0

SCS - Straight Cable System

Задание Ethernet-кабеля



синхрониз.
"перед кучей"

MAC
получ.

MAC
отправ.

контроль
суммы
для проверки

MAC-адрес

3 байта

3 байта

используются
устройства

00:11:22:33:44:ab

OUI - org. unit. id
идентификатор производителя

универс. число в
серии производителя

универсальный MAC-адрес: ff:ff:ff:ff:ff:ff

Поле T/L: мин-указание о продолжении

если T/L < 1536 - это дано в 8 байт

если T/L > 1536 - это мин

Ethernet исп. **CSMA/CD** : процесс. выявления
 через обнаружение
 после обнаружения сигнала свой цикл:
 если произошла коллизия
 - повтор на свой. время
 если второе попытка неудачна,
 повтор увелич. $\times 2$
 всего 16 попыток, далее очередь
 отбрасывается

Макс. длина кабеля Ethernet: **1500 м**
 Мин. длина: **64 м**

Удобств. коммутаторов: **мощь**

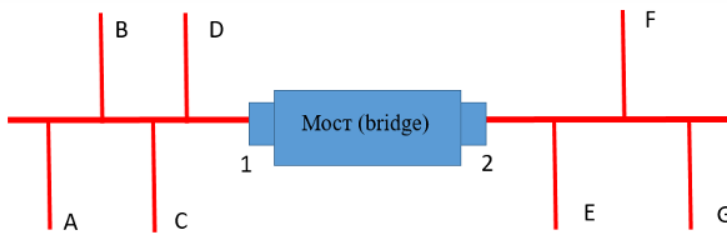
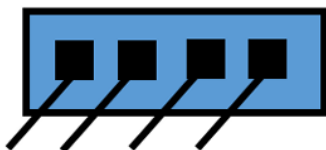


Рисунок 43. Мост – устройство канального уровня.

Мост запоминает, через какой из интерфейсов он видел трафик от того или иного MAC-адреса. Мост анализирует проходящий трафик и строит мостовую таблицу, то есть таблицу коммутации, в которой есть следующие поля: MAC-адрес, порт и время (см. в таблице ниже (Таблица 5)).

Коммутатор (Switch) – многопортовый мост (то есть интерфейсов не два, а больше). Как правило, интерфейсов бывает 8, 16, 24 или 48. Коммутатор аналогичным образом строит мостовую таблицу. За счёт увеличения числа портов уменьшается размер сегмента сети, подключённого к интерфейсу коммутатора.



=> число коллизий меньше

определённый

MAC	Port	Time
MAC-A	1	t ₁
MAC-D	1	t ₂
MAC-E	2	t ₃

=> увелич. пропускной

+ уменьшение
 коллизий
 домена

Коллизия домен:
 число сегм., в кол.
 расширять коллизии

Микросегментация – разбиение коммутатором одного домена коллизий на меньшие коллизийные домены.

Полудуплекс - в один момент времени
передает в одну сторону

Дуплекс - возм. одновр. передачу в обе
стороны

Решение проблем коммутирования

• Store and Forward

К - коммутирование

- 1) К сохраняет пакет
- 2) проверяет контрол. сумму
- 3) если сумма ок, то К. имеет право
и отправляет

сеть
надежная

• Cut through

- 1) К получает пакет и сразу отправляет
пакет (принимает и отправляет
одновременно)

сеть
ненадежная

замеч.: если эк. интерфейс занят
или сл. эк. порт. Выход принят,
К сможет собрать пр. пакет и отправить.

• Fragment free

- 1) К принимает первый 64 бита
- 2) если коллизия нет, К отправляет пакет

идет из предыдущих решений

Сетевой уровень

8	24
16	16
24	8
32 бита	

- класс А;
- класс В;
- класс С;

	— хостовая часть;
	— сетевая часть.

сетевой ч. совмещу.
успр. в одной подсети

Сетевая маска — конструкция длиной 32 бита, записанная в строго определённом формате — определяет, каким образом IP-адрес делится на сетевую и хостовую части. Сна-

11111111111111111111111111111111	0000	— маска
0110110111011111110101110001	1111	— IP-адрес
Сетевая часть		Хос- товая часть

Мы видим, что деление на сетевую и хостовую часть произошло **не по границе байта** (в классовом подходе это было невозможно). Такой подход к делению сетей с использованием масок называется **VLSM³⁸** (variable length subnet masks). Изменяя хостовую часть, можно получить разные

- Ещё в хос. ч.:
- все нули
это одна подсеть
 - все единицы
это широковещ. в подсети
 - n бит
в сети $2^n - 2$ адресов

IPv4 заголовок

32 бита

Version	IHL	Type of Service	Total Length	
Identification			Flags	Fragment Offset
Time to Live	Protocol		Header Checksum	
Source Address				
Destination Address				
Options				Padding

- Version (4 бита) — версия протокола IP (IPv4/IPv6);
- IHL — Internet Header Length (4 бита) — длина заголовка IP-пакета (измеряется в 32-битных словах);
- DSCP — Differentiated Services Code Point (6 бит) — указывает на то, что за трафик передаётся и как его нужно обслуживать;
 - ECN — Explicit Congestion Notification (2 бита) — возможность явного уведомления о перегрузках. DSCP и ECN используются для обеспечения качества обслуживания³⁹;
- Total Length (16 бит) — длина IP-пакета в байтах ($\leq 2^{16}$);
- ID — Identifier (16 бит) — идентификатор пакета;
- Flags (3 бита) и Fragment offset (13 бит) — смещение фрагмента: эти поля отвечают за фрагментацию пакета — дробление на более мелкие части;
 - TTL — Time To Live (8 бит) — время жизни пакета (измеряется в хопсах) — при переходе через маршрутизатор TTL уменьшается на единицу. Если TTL = 0, то пакет отбрасывается;
- Protocol (8 бит) — в этом поле в явном виде указывается тот протокол, данные которого инкапсулированы в пакет (например, TCP/UDP/ICMP);
- Header Checksum — контрольная сумма заголовка пакета (поле данных не используется при расчёте)⁴⁰;
- SA — Source Address — IP-адрес отправителя;
- DA — Destination Address — IP-адрес получателя;

Интерфас маршрутизации: либо первые, либо последние бита в маске

Давайте проверим, какая из записей в таблице ниже (Таблица 14) соответствует пункту назначения пакета (узлу С).

Таблица 14. Таблица маршрутизации для проверки пакетом до узла С.

1	C	192.168.1.0/24	11000000	10101000	00000001	00000000
2	C	192.168.12.0/24	11000000	10101000	00001100	00000000
3	C	192.168.13.0/24	11000000	10101000	00001101	00000000
4	S	192.168.2.0/24	11000000	10101000	00000010	00000000
5	S	192.168.3.0/24	11000000	10101000	00000011	00000000
		192.168.2.2 (адрес С)	11000000	10101000	00000010	00000010

Еще раз повторим все вышесказанное: **сначала выбираем строки, в которых сетевая часть совпадает с той же частью в адресе получателя** (если маска /24, то первые 24 бита сети и адреса получателя должны совпасть), **а потом среди них отбираем строку с самой длинной маской.**

Чтобы решить эту проблему вводится следующее понятие: **AD (administrative distance)** – число, определяющее степень доверия источнику маршрутной информации. Чем меньше значение AD, тем более доверенный источник. Значения AD по умолчанию в маршрутизаторах Cisco для разных типов источников маршрутной информации:

- C (connected) – то, что подключено напрямую – AD = 0 [самые доверенные];
- S (static) – те записи, которые администратор вносит вручную – AD = 1;
- OSPF⁴⁴ – гибридный протокол динамической маршрутизации – AD = 110;
- RIP⁴⁵ – дистанционно-векторный протокол динамической маршрутизации – AD = 120;
- BGP⁴⁶ – path-vector протокол динамической маршрутизации – AD = 20 или 200 (зависит от типа маршрута).

Если разные источники маршрутной информации попытаются отправить в таблицу маршрутизации одинаковые префиксы, то будет выбран источник с наименьшим AD и именно эта запись попадет в таблицу маршрутизации. Администратор может намеренно поменять значение AD для каких-то источников маршрутной информации.

Применение

Введём новый термин. **MTU** (**Maximum Transmission Unit**) – это максимальный объем данных, который может быть отправлен по сети без дальнейшего фрагментирования (одним пакетом).

Фрагментация⁴⁹ – деление исходного IP-пакета на два или более фрагмента, так, чтобы в итоге эти фрагменты были меньше, чем MTU на канале.

Все пакеты с одинаковым ID явл. фрагментами одного пакета.

DF (don't fragment) – если этот флаг поднят (его значение = 1), то отправитель пакета запретил выполнять фрагментацию пакета (если пакет не пролезает – отбрасываем).

MF (more fragments) – указание на то, является ли наш фрагмент последним в цепочке фрагментов. То есть если MF = 0, то это последний фрагмент. Этот флаг помогает правильно собрать фрагменты. В исходном (не фрагментированном) пакете этот флаг равен 0 (так как это единственный пакет).

Fragment Offset – это поле, которое позволяет понять место расположения данного фрагмента относительно начала исходного IP-пакета. В каждом фрагменте мы вы-

ARP

ARP⁵¹ – протокол, который позволяет в локальном сегменте (то есть там, где для передачи информации можно обойтись без роутера) по известному IPv4 адресу вычислить MAC-адрес.

← работаем
внутри
сети

Узел А знает IP узла В по DNS.

Для того чтобы узнать MAC-адрес В,

А рассылает запросы. **ARP-request** :

«Узел В, имеет у
тебя MAC-адрес?
и если да, то какой»

Технология, позволяющая исправить такую ошибку

Proxy ARP.

Эта технология включается на маршрутизаторах и работает следующим образом: если какой-то роутер на одном из своих интерфейсов видит ARP-request об IP-адресе, который не может существовать в этом сегменте, а роутер знает маршрут до этого адреса, то роутер на этот ARP-request ответит сам, своим MAC-адресом.

ICMP - служебный протокол сетевого уровня

Функции:

- ping и echo-request для проверки доступности узла
- уведомление, что пакет отброшен из-за превышения DF

=> механизм Path MTU discovery:

поиск мин. MTU, при ком. пакет будет до конечн. узла

- уведомление, что пакет отброшен из-за превышения TTL

=> механизм Tracerouting:

постепенно увеличивая TTL на единицу, начиная с 0, можно узнать IP всех маршрутизаторов, через которые проходит пакет

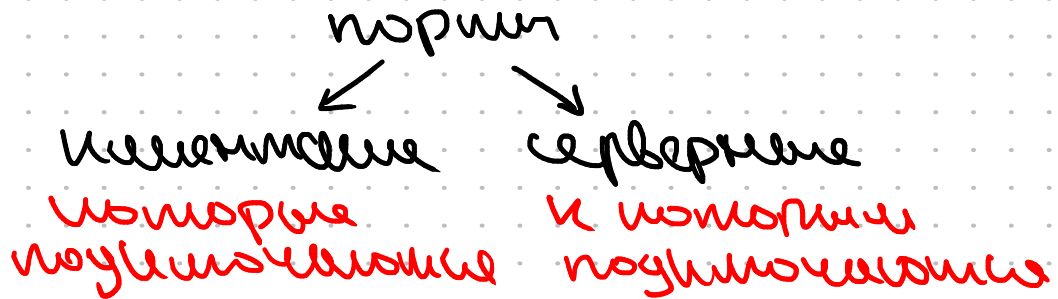
- уведомления о безопасности (атаках) сети

UDP

порты от 1 до 65535

Биты	0 - 15	16 - 31
0-31	Порт отправителя (Source port)	Порт получателя (Destination port)
32-63	Длина датаграммы (Length)	Контрольная сумма (Checksum)
64...	Данные (Data)	

мем. для
данных
= 2^{16}



Первые 1000 портов - известные,
зарезервированы для клиентских сервисов
Остальные - не зарезервированы
1 порт - 1 процесс
1 процесс - много портов

Socket - пара: IP-адрес + Port
Получение процесс. адреса 2 способами

Итак, порт идентифицирует процесс на хосте, а
IP-адрес идентифицирует хост глобально, значит Socket
идентифицирует процесс глобально.

TCP

если возн. переопределения
доследи

мем. 65535
портов

0 - 3	4 - 6	7 - 15	16 - 31
Порт источника, Source Port			Порт назначения, Destination Port
(1) Порядковый номер, Sequence Number (SN)			
(2) Номер подтверждения, Acknowledgment Number (ACK SN)			
(3) Длина заголовка, (Data offset)	Зарезервировано	(4) Флаги	Размер Окна, Window size
Контрольная сумма, Checksum			Указатель важности, Urgent Point
Опции (необязательное, но используется практически всегда)			
Данные			

- (1) Sequence Number — номер 1-го байта в сегменте
- (2) Ack. Number — до какого номера данные были получены.
- (3) Data Offset — указывает, где начался сегмент, начиная с начала
- (4) Флаги:

URG — важно/не важно

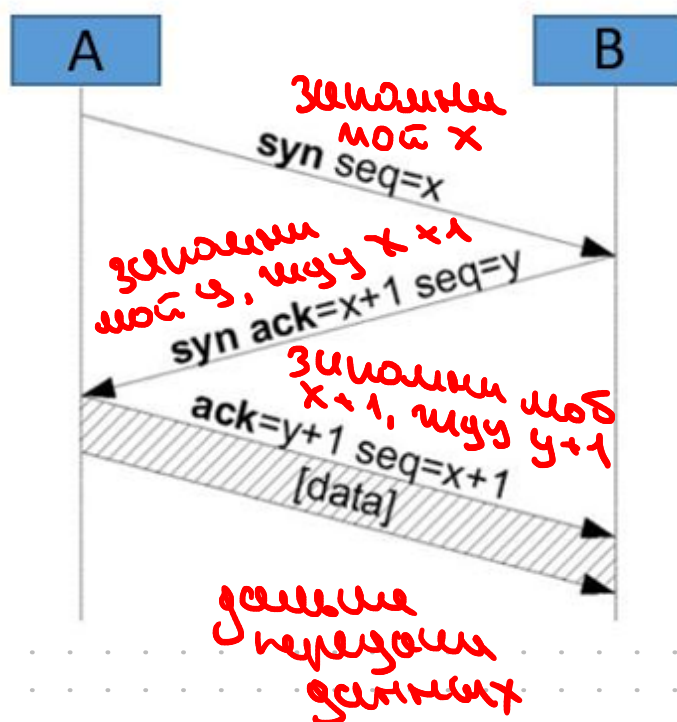
ACK — номер и не Ack. Num.

PSH — данные не нужно хранить в буфере (=1)
push

RST и FIN — где завер. сессии
reset *finish*

SYN — где возьмем seq. Num.

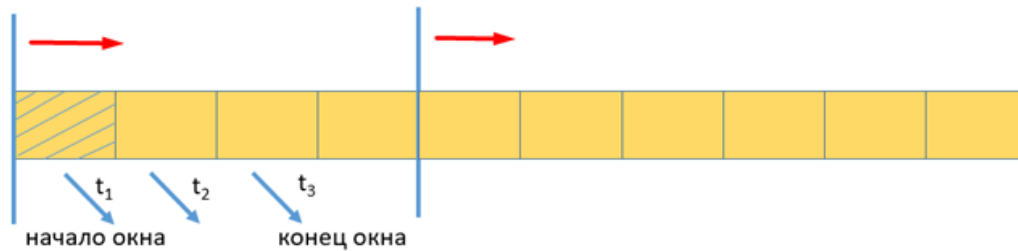
Трехное рукопожатие



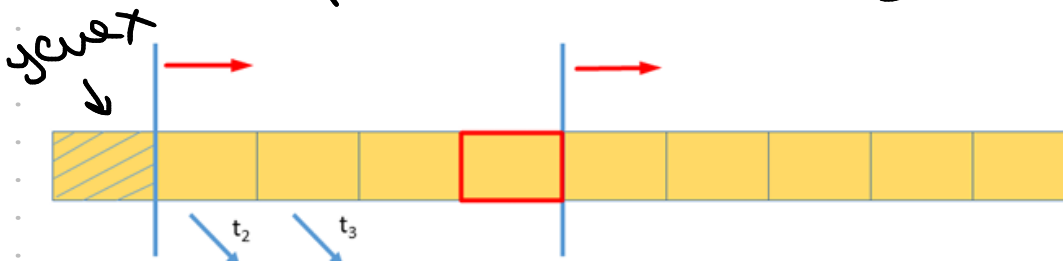
В ответ на seq. $x+d$, где d — число байт

Скользящее окно

Механизм скользящего окна — отправление нескольких сегментов сразу (пачкой) до получения подтверждения их получения.



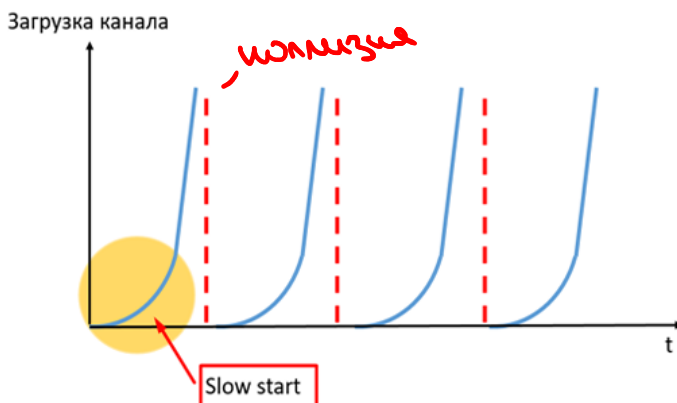
сдвигаем окно вправо, если левые кадры успешно переданы



То есть клиентская сторона может отправлять данные быстро, а принимающая сторона не успевает принимать данные с такой скоростью (не успевает записывать). Поэтому на принимающей стороне **необходим механизм по регулированию той скорости, с которой данные к ней поступают: window size.**

Поскольку в TCP передача двунаправленная, то каждая сторона изменяет окно у своего соседа: говорим противоположной стороне размер окна, который использовать для отправки сообщений в нашу сторону. Отметим также, что сторона-отправитель может сама изменить размер используемого окна, но только в меньшую сторону. То есть если отправитель понимает, что у него по какой-то причине нет ресурсов отправлять данные с какой-то скоростью, он может сам уменьшить размер окна.

Slow start



Оптимизация TCP

1. Задерживающее подтверждение

Внесено предложение, чтобы узел B опирившаяся АСН о получение большого сегмента, узел B может получить несколько сегментов и опирившаяся АСН о них вместе.

2. Алгоритм Хейла

Внесено предложение, чтобы составить сегменты из меньшего количества сегментов, Алгоритм Хейла задерживает опирившуюся и составляет сегменты из большего числа сегментов.

3. Quality of Service

Поле QoS - это 2 бита, которые определяют:

CWR и ECN-Echo

Их можно использовать для увеличения оборудования для увеличения о перегрузке (мин. перегруз. буфера)

(опция)

4. MSS (Max. Segment Size)

$MSS = MTU - (IP - \text{заголовок}) - (TCP - \text{заголовок})$

В заголовке TCP в поле Options есть MSS. MSS - макс. размер полезного бита данных в сегментах для сегмента.

Во время первого рукопожатия клиент может изменить MSS в сообщении SYN => MSS меньше, чем сегменты перегружены без подтверждения

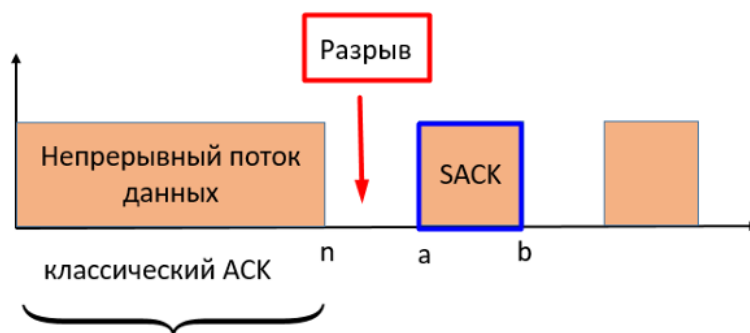
(опыт)

5. **SACK/NACK** - подтверждение данных
и разрывами

selective ack
negative ack

- SACK - подтверждение данных после разрыва (редис. сети.)

В поле Options можно указать, что уже получены данные [0, n] и [a, b], где a и b - границы блока данных, пришедшего после разрыва



- NACK - в поле Options указывается, что данные (n, a) не дошли

(опыт)

6. **Scaling Factor** - еще один пункт в Options

Для малых (быстро меняется),
средних (быстро RST)
и больших (малые потери)
меньше задержек и т.д.

Scaling Factor где увеличивается
время прослушивания канала.

Scaling Factor хранит число n
где увеличение скорости сети:

$$WS^x = 2^n \cdot WS$$

7. Быстрое повторное определение

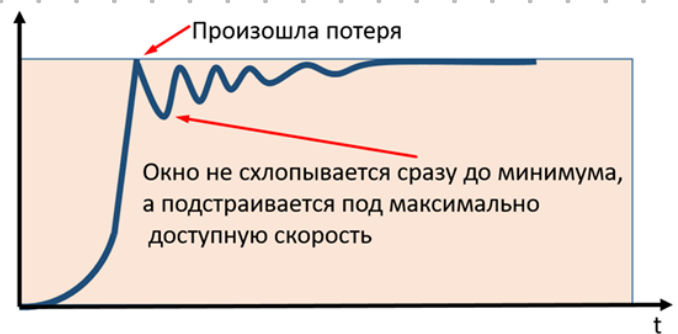
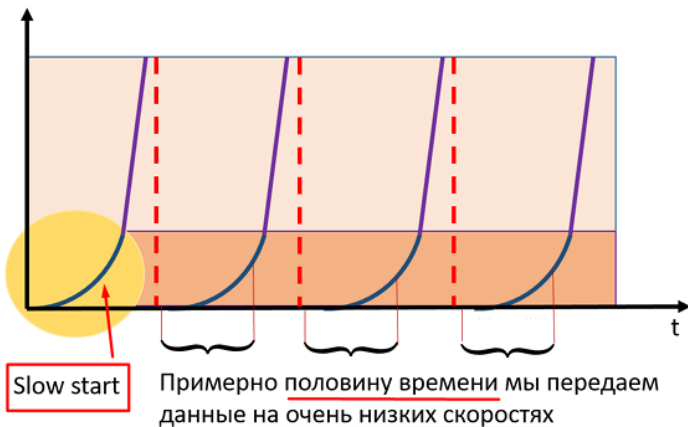
Fast Retransmission

- 1) Получатель не принял кусок данных
- 2) Получатель быстро определяет при ACK'a о получении данных до ^{- кусков}
- 3) Отправитель, получив 3 ACK'a не увеличивает номер и сразу перешит данные

8. Регулировка скорости потока

• Классика: Slow Start

• Современная реализация



9. SYN cookie - защита от SYN Flood (сильно попытками открыть сокет)

- 1) Сервер понимает, что к нему пытаются подключиться злоумышленники (или просто пользователи)
- 2) Сервер выш. SYN cookie: он получ. SYN и вместо обычного ответа отправляет соед. SYN+ACK, но в нем Sequence Numb запис. хэш от IP отправителя, порт и т.д. не откр. сокет!
- 3) Если запис. о подключ. без проблем, сервер получ. хэштег и далее откр. сокет

10. TFO-TCP Fast Open

- 1) Впервые в процессе пробного рукопожатия сервер доверяет клиенту с клиентом о значении cookie - уник. числ
- 2) При повторных подключениях клиент сразу отправляет cookie серверу
- 3) Сервер "узнает" клиента и немедленно пересылает данные до завершения рукопожатия.

DHCP

протокол для динамического назначения сетевых параметров на оборуд.

• Процедура получения IP-адреса

Discover
Это сообщение, которое отправляет клиент широковещательно = клиент не знает, есть ли в сети DHCP-сервер = пытается его найти. У клиента пока нет IP-адреса [сообщение отправляется с адреса 0.0.0.0 на адрес 255.255.255.255].
Offer
Получив сообщение Discover от клиента, сервер отправляет ему сообщение Offer = предлагает клиенту IP-параметры. Вместе с IP-параметрами DHCP-сервер в сторону клиента отправляет информацию о времени аренды = сообщает, в течение какого времени ⁷⁷ клиент может использовать эти параметры.

Когда проходит половина времени аренды, клиент должен запросить продление = отправить Request = «хочу продолжать использовать эти параметры». Сервер может подтвердить продление (Acknowledgment) или, если он не может продлить использование, не подтвердить (Negative Acknowledgment). Может случиться ситуация, что у нас вышел из строя DHCP-сервер. Прошла половина времени аренды, клиент отправляет Request = пытается продлить аренду, но никто не отвечает. У клиента остается половина исходного времени аренды, он ее опять делит пополам и по-

Request
Клиент, получив IP-параметры, если они его устраивают, хочет закрепить за собой право их использовать и отправляет на сервер Request [сообщение отправляется с адреса 0.0.0.0 на адрес 255.255.255.255, это делается для того, чтобы уведомить остальные DHCP-сервера в данном сегменте о том, какой именно адрес предпочел клиент, чтобы сервера могли раньше освободить временно зарезервированный под данного клиента адрес].
Acknowledgement
Сервер, если он не передумал, отправляет клиенту Acknowledgement = говорит, что он зарезервировал эти параметры за клиентом: IP-адрес, маску, возможно, адрес шлюза по умолчанию и другие.

том повторно отправляет Request (через $\frac{3}{4}$ от исходного времени аренды⁷⁸). Если сервер клиенту так и не ответил, клиент по истечении времени аренды освобождает IP-адрес и начинает процедуру получения заново.

• Другие сообщ.

Release
Release = клиент освобождает IP-адрес. Сервер возвращает этот адрес в пул свободных адресов и сервер может через какое-то время выдать другому клиенту.
Request
Клиент, когда-то получал адрес, использовал его и потом отключился от сети. Вернувшись, он пытается запросить право на использование этого же адреса. Сервер может не подтвердить такой запрос и отправить клиенту Negative Acknowledgment. Например, адрес уже выдан кому-то другому.

Decline
Рассмотрим ситуацию, когда мы получили IP-адрес от DHCP-сервера. Но, прежде чем начать его использовать, мы должны проверить, не использует ли его кто-нибудь другой. Это необходимо делать, несмотря на то, что у серверов есть своя проверка на занятость адресов. Дело в том, что эта проверка у сервера не всегда корректная = сервер может посчитать адрес свободным, в то время как в реальности этот адрес кем-то используется ⁷⁹ . Рассмотрим, как клиент проверяет, не занят ли адрес, предложенный сервером. Клиент отправляет GARP-сообщение (Gratuitous ARP) = клиент рассылает ARP-запрос о своем собственном IP-адресе = проверка, есть ли в этом сегменте сети устройство, на котором назначен такой же адрес. Если этот адрес не используется, то нам никто не ответит. Но если есть устройство с таким адресом, то мы по понятным причинам не можем его использовать. В такой ситуации мы должны отправить серверу сообщение Decline = отказываемся от предложенного адреса.
Inform
Используется, когда устройство уже получило IP-адрес и другие настройки через DHCP, но нужно запросить у сервера дополнительные параметры сети, не меняя при этом IP

Проблема: ширковелу. Discover не проходит через маршрутизаторы
Верным решением:

1. Если маршрутизатор DHCP-сервером не является, то можно назначить его на роутере

2.

IPAM

— система, которая в одном месте хранит информацию о том, какие сети для каких целей использованы, какие в этих сетях ресурсы, сколько адресов выдано (какой % утилизации адресного пространства в той или иной подсети). То есть сервера с функцией IPAM могут сказать, какие сегменты близки к истощению. Таким образом, у нас появляется центральная точка, которая позволяет управлять адресным пространством.

3. **DHCP Relay**

Роутеры принимают DHCP-сообщения, но вводят их, чтобы выдать IP-адреса, они направляют сообщ. дальше и центрируются между DHCP-серверами

4. DHCP Option 82

Иногда хотим иметь адресированный IP
Все зависит от усур-во, с кот. он подкл.

DHCP Option 82 — компьютер вставляет
в сообщ., которое придет от клиента адрес 82
и говорим, с какого усур. проводим
игра.

=> отдаем адрес IP на компьютер провод
(комп. порт компьютера)

RIP

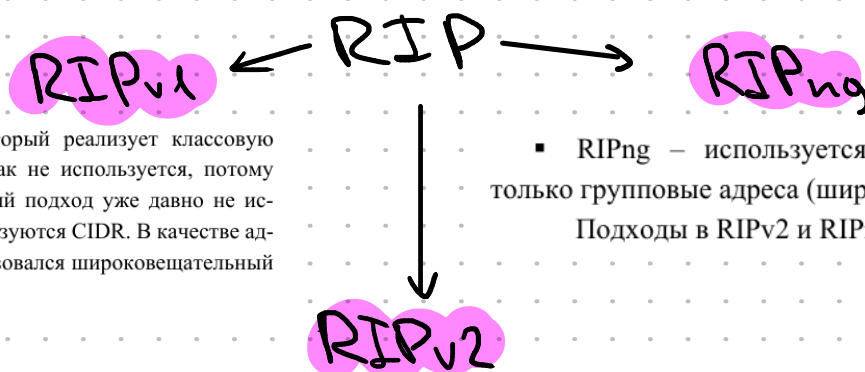
RIP (Routing Information Protocol) — протокол динамической маршрутизации. Это один из наиболее простых и медленных протоколов динамической маршрутизации (имеется в виду его скорость конвергенции⁸⁰), который при этом до сих пор поддерживается и используется.

Время конвергенции — время сходимости сети, то есть интервал времени, необходимый для того, чтобы все маршрутизаторы отреагировали на изменения в сети.

Этот протокол использует **дистанционно-векторные алгоритмы**. То есть роутер, на котором запущен RIP, отправляет своим соседям маршрутную информацию: **префикс + метрика**.

Метрика — число, которое характеризует удаленность того или иного префикса = насколько далеко до той или иной подсети. Чем больше метрика, тем хуже — тем дальше эта сетка расположена от нас.

— обычно,
метрика — число
роутеров
в пути
max. метрика 15



▪ RIPv1 — протокол, который реализует классовую маршрутизацию. Сегодня никак не используется, потому что, как мы помним, классовый подход уже давно не используется, вместо него используются CIDR. В качестве адреса получателя всегда использовался широковещательный адрес — 255.255.255.255;

▪ RIPng — используется в сетях IPv6. Использует только групповые адреса (широковещательных нет). Подходы в RIPv2 и RIPng примерно одинаковые.

▪ RIPv2 — используется в сетях IPv4. В качестве адреса получателя может также использоваться широковещательный адрес (broadcast) или групповые адреса (multicast);

Таймеры в RIP

Update timer (30 секунд) – частота отправки сообщений с обновлениями. По истечению данного таймера отправляется обновление. Каждые 30 секунд RIP рассылает обновления обо **всех** префиксах, которые находятся в базе данных RIP. Соответственно, если роутер знает много префиксов, то каждые 30 секунд он отправляет много данных.

Invalid timer (180 секунд) – если обновление о маршруте не будет получено до истечения данного таймера, маршрут будет помечен как invalid (недостижимая сеть), то есть сеть с метрикой 16. Другими словами, если произошел отказ какого-то канала/соседа, то роутер 180 секунд все равно считает эту сеть достижимой и в этом направлении отправляет данные (не происходит перестроение сети). **Invalid timer перезапускается после получения каждого обновления, содержащего информацию о выбранном префиксе, и если за 180 секунд мы успели получить новое, то он сбрасывается.**

Если для какого-то префикса истек Invalid timer, то в этот момент префикс не удаляется из таблицы маршрутизации, а остается в ней с метрикой 16. И в этот же момент запускается Hold down timer.

Hold down timer (180 секунд) – запуск таймера произойдет после того, как маршрут был помечен как недостижимый. До истечения данного таймера маршрут будет находиться в таблице маршрутизации. То есть роутер в течение 180 секунд каждые 30 секунд отправляет соседям информацию о том, что этот префикс недостижим = «замораживает» префикс, сообщая, что через него этот префикс недостижим. Это позволяет избежать возникновения петель в переходные моменты (то есть, когда сеть «сходится»). Если по истечению Hold down timer какой-то из соседей продолжает нам говорить, что он знает маршрут, который мы ранее помечали как недостижимый, то мы понимаем, что это не петля, не эффект сходимости сети, а он действительно знает маршрут **не через нас.**

Flush timer (240 секунд) – перезапускается после получения каждого обновления. По истечении этого таймера информация о префиксе удаляется.



Оптимизации RIP

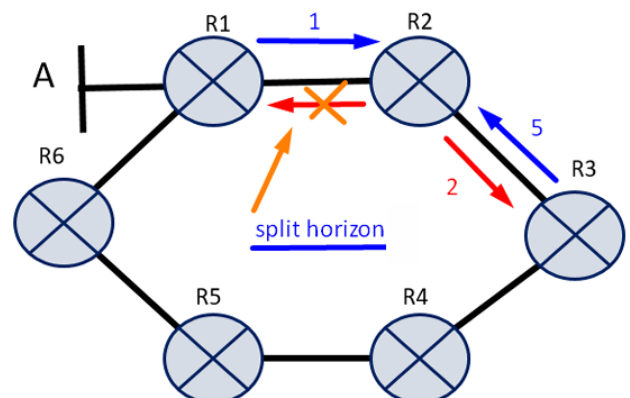
1) Решение горизонти Split Horizon

▪ **Split horizon** – это оптимизация, в которой роутер не отправляет обновления обратно через тот интерфейс, через который он их получил, **если он их использует.** Рассмотрим

=> не отправляем назад данные, от которых мы их получили!

2) Poison reverse

▪ **Poison reverse** (усиление оптимизации split horizon) – в сторону интерфейса, от которого мы получили **лучший маршрут**, мы обратно в обновлении отправляем этот же префикс с метрикой 16, то есть **помечаем его как недостижимый.** В нашем примере R2 отправляет R1 обновление, что метрика до сети A равна 16 (см. Рисунок 99).



3) Route poisoning

▪ **Route poisoning** – в рамках этой оптимизации, когда мы понимаем, что какая-то сеть недостижима, то не просто исключаем этот префикс из обновлений, а начинаем отправлять обновления об этом префиксе с метрикой 16 (недостижимый маршрут).

– чтобы узнать не нужно Invalid timer

4) Триггерное обновление Trigger update

▪ **Triggered update** – обновления отправляются сразу при изменении маршрута вместо того, чтобы ожидать, пока истечет Update timer. Например, у роутера R1 включилась

– не нужно 30 сек через отправляет

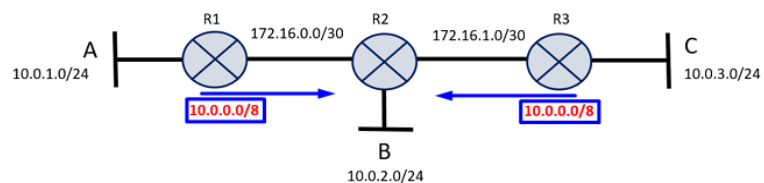
5) Triggered extensions

Triggered extensions – в первом обновлении передается вся маршрутная информация и обновления больше каждые 30 секунд не отправляется. Обновления могут быть отправлены, только если произошло какое-то событие (например, к роутеру R1 подключили сеть C). Это решение позволяет каналу быть выключенным в течение какого-то времени

6) Суммаризация и ESRP

ESRP (Equal Cost MultiPath) – балансировка по множеству путей с одинаковой стоимостью. Таким образом примерно 50% пакетов будет отправляться не туда (в сеть A придет только половина отправляемых данных, вторая половина уйдет в сторону сети C). Чтобы избежать этой проблемы при настройке RIP необходимо **отключать автоматическую суммаризацию**. В этом случае роутеры R1 и R3 будут отправлять R2 настоящие маршруты 10.0.1.0/24 и 10.0.3.0/24, а не пытаться агрегировать их до размера классовой сети.

→ Суммаризация необходима, чтобы уменьшить количество префиксов, которые передаются (распространяются) по сети, уменьшить размер таблицы маршрутизации (RIB – Routing Information Base).



Сети 10.0.1.0/24, 10.0.2.0/24 и 10.0.3.0/24 изначально принадлежали классу A (первый бит 0).

Сети 172.16.0.0/30 и 172.16.1.0/30 изначально принадлежали классу B (первый бит 1, второй 0).

Таким образом, роутер R1 соединяет разные классы сети. По умолчанию, роутер, на котором работает RIP, выполняет автоматическую суммаризацию до сети класса. Это означает, что R1 в сторону R2 будет отправлять обновления не о сети 10.0.1.0/24, а о классовой сети (10.0.0.0/8). Аналогичная ситуация происходит и справа у роутера R3 (см. Рисунок 102).

Таким образом, если из сети B нужно будет отправить данные в сеть A, то роутер R2 столкнется с неразрешимой задачей: он не будет знать, куда ему отправить эти данные. R1 и R3 объявляют роутеру R2 одинаковые префиксы, оба префикса объявляются через RIP и имеют одинаковый administrative distance (AD в RIP по умолчанию – 120). Таким образом, у R2 есть два равноправных маршрута, которые имеют одинаковую метрику (и слева, и справа метрика = 1), а значит, оба попадут в таблицу маршрутизации.

▪ Ручная суммаризация в RIP

В этой ситуации мы сами хотим выполнить агрегирование до какого-то префикса. Например, у нас есть сеть 10.0.1.0/24, мы можем выполнить агрегирование и отправлять в сторону R2 маршрут до сети 10.0.0.0/16 (см. Рисунок 102). То есть мы указываем в обновлении сеть большую, чем у нас реально есть (у нас сеть /24, а мы отправляем /16: в нее попадает наша /24 сеть).

NAT

NAT (Network Address Translation) – трансляция сетевых адресов – технология, которая позволяет маршрутизатору заменить IP-адрес отправителя и/или IP-адрес получателя в заголовках IP-пакетов, которые проходят через него.

• Статический NAT

В этом случае на роутере указывается, что будет проходить трансляция одного конкретного адреса.

— изменяем 1-й адрес

• Динамический NAT *необходимо не использовать.*

Это решение, которое применяется для трансляции группы адресов. То есть роутер может группу «серых» адресов вашей локальной сети транслировать в адреса из подсети, которая имеет выход к глобальным ресурсам



Провайдер, из-за нехватки IPv4 адресов не может выделить нам /24 сетку адресов, но готов предоставить более мелкий диапазон, например, /26. В этой ситуации мы не можем воспользоваться статическим NAT: нет однозначного соответствия между диапазоном адресов нашей локальной сети и тем диапазоном, который нам предоставил провайдер ($2^8 > 2^6$). Однако, скорее всего, в нашей сети не все компьютеры работают одновременно: часть устройств может быть выключена. В этой ситуации можно поставить в соответствие работающим узлам из диапазона /24 адреса из пула, предоставленного провайдером (/26). Это сопоставление происходит динамически: пока устройство нашей сети включено и отправляет какой-то трафик, роутер производит трансляцию (подменяет адрес отправителя). После выключения компьютера запись из таблицы NAT-трансляций будет удалена, а «белый» адрес из пула провайдера возвращен в список адресов, которые можно использовать для трансляции.

PAT

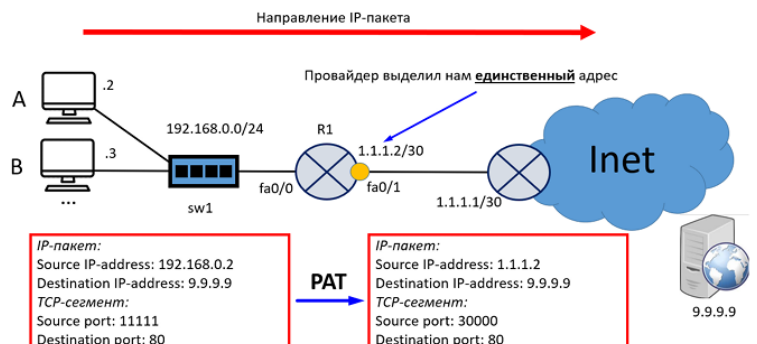
PAT (Port Address Translation) – трансляция адресов с использованием портов транспортного уровня с которых и/или на которые устанавливается соединение между процессами на устройствах.

порт описывается ч.с. изменяем

• Динам. PAT

Эта технология вмешивается в работу третьего и четвертого уровней модели OSI. Технология PAT кроме трансляции IP-адреса производит также трансляцию порта (TCP/UDP). Допустим, в Интернете расположен

Когда трафик пойдет из локальной сети в глобальную, произойдет изменение не только IP-адресов, но и номеров портов. IP-адрес получателя останется без изменений, а IP-адрес отправителя R1 изменит на 1.1.1.2. То есть

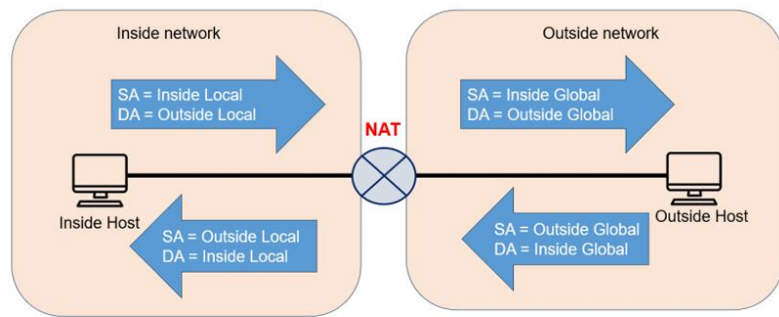


Inside Local – адрес хоста в локальной сети, когда пакет находится также в локальной сети.

Inside Global – адрес хоста в локальной сети, когда пакет находится во внешней (глобальной) сети.

Outside Local – адрес хоста в глобальной сети, когда пакет находится в локальной сети.

Outside Global – адрес хоста в глобальной сети, когда пакет находится в глобальной сети.



• Симметричный NAT

Этот подход предполагает, что мы вручную создаем запись для таблицы NAT-трансляций. Рассмотрим ситуации, для которых это может потребоваться.

Пример 1. Необходимо опубликовать какой-то сервис из локальной сети во внешний мир.

Пример 2. Необходимо разместить в локальной сети HTTP-сервер (на узле A), у которого будет 2 процесса: первый процесс будет обслуживать пользователей локальной сети по 80 порту, второй – предназначен для клиентов из Интернета и доступен по порту 8080 (или еще какому-то другому, не 80). Мы хотим, чтобы пользователи локальной

Решения работы NAT

1) Symmetric

Для обработки вход. узлов нужно полное соответствие адрес и портов отправ./получ. Число портов будет ограничено.

2) Full Cone

Во входных (an Global) пакетах проверка только по адресу, адрес и порт известны. Адрес и порт источника не имеют значения.

3) Address Restricted Cone

Важен адрес, не важен порт.

4) Port Restricted Cone

Важен порт, не важен адрес

FTP и NAT

В FTP

есть управляющая сессия (control-session) и есть подключение, по которому передаются пользовательские данные (data-session).

Последовательность подключений выглядит так: устанавливается control-session и клиент сообщает через неё что он собирается сделать: скачать какой-то файл или посмотреть список файлов в директории и так далее. После того, как клиент отправил соответствующие команды, устанавливается новое (второе) соединение, и через него эти данные передаются.

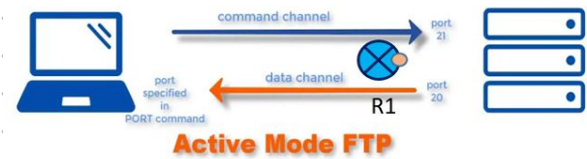
В активном режиме
появл. проблемы, тк.
сервер уст. подкл. и откр. порт,
но NAT/PAT то не пропускает
и клиенту

Решение: **ALG**

ALG – это специальный механизм, который работает на маршрутизаторе и анализирует трафик уровня приложений, то есть заглядывает на 7-ой уровень (OSI) и смотрит, какими FTP-командами обмениваются клиент и сервер

Роутер с технологией ALG видит, что клиент в сторону сервера сообщает номер порта (порт отправителя со стороны клиента, см. Рисунок 103) и роутер понимает, что теперь сервер попытается подключиться по второй сессии на этот порт. В результате роутер может на себе создать динамическую трансляцию для того, чтобы новое подключение со стороны сервера было перенаправлено на корректный порт правильного клиента. Таким образом, роутер анализирует то, что передается в сообщениях протокола уровня приложений и сам создает необходимую NAT/PAT-трансляцию.

FTP построен на архитектуре «клиент-сервер» и умеет работать в одном из двух режимов: активном или пассивном. Обозначим клиента – C, а сервер – S на рисунке (Рисунок 110).



Первая сессия (control-session) и в активном, и в пассивном режиме устанавливается со стороны клиента в сторону сервера. Если FTP работает в активном режиме, то сессия с данными (data-session) будет устанавливаться со стороны сервера в сторону клиента. В пассивном режиме сессия с данными будет устанавливаться со стороны клиента в сторону сервера.

Минус ALG

- 1) замедленно
- 2) не для всех приложений
- 3) сейчас весь трафик зашифрован

⇓
UPnP

Если клиентское приложение на себе открывает какой-то порт, то UPnP в сторону роутера отправляет соответствующее сообщение: клиент сообщает роутеру, что он на себе открыл такой-то порт. Роутер тогда добавляет соответствующую запись в таблицу PAT-трансляций и правило в межсетевой экран, позволяя внешнему трафику попадать внутрь локальной сети.

В работе данной технологии есть ряд ограничений. Например, если два роутера расположены подряд (друг за другом в цепочку), этот механизм не работает по причине того, что UPnP работает только между клиентом и ближайшим маршрутизатором.

IPv6

IPv6 адрес представляет собой конструкцию длиной в 128 бит, записанную в 16-ричной форме. Весь адрес делится на группы по два байта (два байта = четыре 16-ричные цифры). Каждая группа отделяется от другой двоеточием. Ниже представлены примеры записей IPv6 адресов:

- 2001:0000:0db8:0000:0000:0000:07a0:765d; (1)
- 2001:0:db8:0:0:0:7a0:765d; (2)
- 2001:0:db8::7a0:765d. (3)

2 байта ↑ т.к. не совсем и т.д. можно 1 раз
через 4 знака: 0000 нулей
(2, 2)

Зарезерв. адреса и маски

- ::/0 – маршрут по умолчанию. Это префикс, который, как и в IPv4, описывает любые адреса и представляет собой запись из одних нулей с маской /0.
- 2001:2345:6789::/64 – адрес стандартной какой-то подсети. В IPv6 вводится понятие стандартной маски – /64. Как и в IPv4, адрес сети состоит из самой подсети (первые 64 бита для стандартной маски = первые 16 символов) и последовательности нулей, которые представляют собой Interface ID (для стандартной маски это последние 64 бита = последние 16 символов). Interface ID – это та часть адреса, которая в протоколе IPv4 называлась хостовой частью. Как видно из примера, последовательность нулей во второй половине адреса можно также представить двойным двоеточием.

Синтез адреса: 164
(16 символов)

2001:0000:0db8:0000:0000:0000:07a0:765d

подсети

Interface ID

(последние 4)

IPv6 адреса

Unicast

Получением
- 1 адрес

Multicast

Получением
- много адресов

Anycast

Получением
данных через
сущ. адрес

Глобальные

Link Local

Unique Local

- Глобальные (2000::/3) – адреса, которые начинаются с префикса 2000::/3 и являются глобально маршрутизируемыми. Эти адреса вы можете встретить в сети Интернет. Диапазон возможных значений для первой группы цифр: от 2000 до 3FFF.

- Unique Local (FC00::/7) – адреса, которые используются в локальных сетях без выхода в Интернет и являются аналогом частных адресов протокола IPv4. Unique Local адреса состоят из двух групп адресов. Одна группа начинается с префикса fc00::/8, вторая – с префикса fd00::/8. Восьмой бит L (local flag) показывает, был ли префикс назначен локально (L=1, fd00::/8), или адрес зарезервирован для будущих применений (L=0, fc00::/8).

- Link Local (FE80::/10) – не маршрутизируемые адреса, которые назначаются на интерфейсы и используются вместе с Глобальным/Unique Local адресом. Link Local адреса используются для служебных целей.

Отметим еще раз, что Link Local адреса являются не маршрутизируемыми, то есть такой адрес мы можем указать в качестве получателя только для устройств из одного L2 сегмента с нами.

Link Local адреса могут быть назначены на нескольких интерфейсах одновременно. То есть на нескольких интерфейсах одного устройства могут быть назначены адреса, принадлежащие одной и той же подсети. В IPv4 такое было невозможно. В IPv6 это возможно, но только для Link Local адресов. Поэтому при отправке сообщения соседнему устройству на Link Local адрес необходимо, кроме адреса получателя, дополнительно указать тот интерфейс, через который эти данные будут отправляться.

Преимущества IPv6

- ✓ Больше адресное ир-во
- ✓ механизм SLAAC позволяет назначать себе адрес, зная только свою часть
- ✓ более развитые адреса позв. строить более выразительную иерархию сетей
- ✓ Описательные иерархические сетевые:

В IPv4 для связи IP и MAC нужны протоколы ARP
В IPv6 внесено ARP и используется NDP (на базе ICMPv6)

Multicast-адреса

Запрошенные
(Solicited)

Назначенные
(Assigned)

Запрошенные multicast адреса формируются («строятся») по IPv6 unicast адресу. А запросы ICMPv6 о MAC-адресе узла (см. Рисунок 111) будут отправляться именно на такой групповой адрес (multicast), соответствующий IPv6 unicast. Этот групповой адрес может относиться и к другим узлам, у которых «похожие» IPv6 unicast адреса.

В протоколе IPv6 (поскольку multicast адрес получателя ICMPv6 запроса зависит от unicast адреса) устройство на уровне сетевой карты анализирует пакет и понимает, может ли этот запрос быть предназначен для него или этот запрос адресован другому устройству. Это позволяет максимально быстро (на уровне сетевой карты) отбрасывать ненужные пакеты и не поднимать их по стеку до центрального процессора⁹⁷.

В IPv4 тоже нельзя, т.к. checksum IP не используется в заголовке, который сетевая карта не имеет возможности проверить

✓ Оптимизации в заголовке

Заголовок IPv6

режим 64 бит

4	8	12	16	20	24	28	32	36	40	44	48	52	56	60	64
Ver.	Traffic Class	Flow Label						Payload Length				Next Header	Hop Limit		
Source Address															
Destination Address															

Same or Similar Fields in IPv4

New Field in IPv6

- Payload Length (16 бит) – длина полезной нагрузки;
- Next Header (8 бит) – указатель на следующий заголовок. Это аналог поля Protocol в заголовке IPv4;
- Hop Limit (8 бит) – время жизни пакета. Аналог поля TTL в заголовке IPv4.

- Ver (4 бита) – версия протокола IP (IPv4/IPv6);
- Traffic Class (8 бит) – указывает на то, что за трафик передается и как его нужно обслуживать. Это поле является аналогом полей DSCP и ECN в заголовке IPv4;
- Flow label (новое поле, 20 бит) – указатель на поток (идентификатор потока). Это поле позволяет отправителю добавлять информацию о потоке и оптимизировать процесс маршрутизации: роутер может быстрее обрабатывать дополнительные заголовки в пакетах одного потока.

В IPv6 сетевик роутер обрабатывает пакеты одного потока индивидуально, разбираясь с заголовками или первым пакетом.

Таким образом, поток = Flow label + IPv6 адрес отправителя;

● Из заголовка IPv6 исключена контрольная сумма (она присутствует в протоколах канального и транспортного уровней, поэтому и было принято решение убрать эту функцию на сетевом уровне). Это означает, что не нужно пересчитывать контрольную сумму: в заголовке IPv4 уменьшение TTL при прохождении через роутер приводило к необходимости пересчитывать контрольную сумму на каждом роутере. В протоколе IPv6 также происходит decrement

● В заголовке IPv6 отсутствуют флаги MF, DF и поле Fragment Offset. Это означает, что IPv6 не поддерживает фрагментацию в том виде, в котором она была представлена в IPv4. Фрагментация все же может быть выполнена, но только на узле-отправителе пакета. Для этого отправитель пакета после заголовка IPv6 добавляет дополнительный заголовок, который и указывает на то, что данный пакет фрагментирован. Сборка после фрагментации по-прежнему происходит только на получателе пакета⁹⁹. То есть в IPv6 фрагментация вынесена в отдельный дополнительный заголовок фрагментации¹⁰⁰ (как и поле Options, которое присутствовало в заголовке IPv4). Отсутствие фрагментации по пути

● Поле Next Header, в отличие от поля Protocol в IPv4 пакетах, позволяет не просто понять, блок данных какого протокола содержится внутри данного пакета, а в явном виде указывает, какой будет следующий заголовок: это, например, может быть заголовок фрагментации, о котором только что шла речь, или заголовок протокола TCP.

Методы назначения адресов в IPv6

- **Stateful DHCP** – аналог DHCP из протокола IPv4.
- **SLAAC¹⁰¹** – механизм, который позволяет назначить IPv6 адрес на интерфейс устройства без какого-либо выделенного сервера.

← с помощью.
Stateless DHCP
gns, dhcp, DNS-серверов

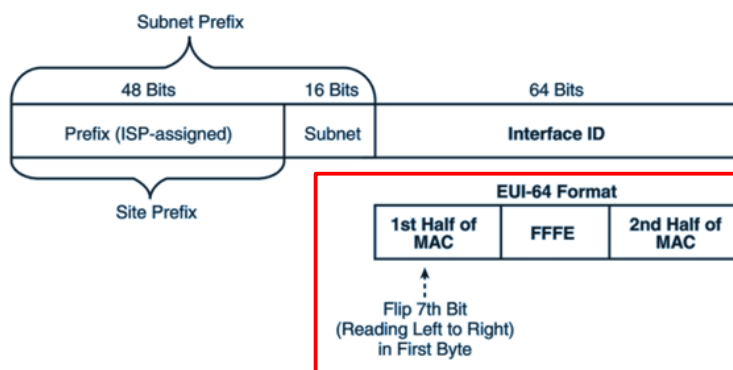
✓ Получение сетевой части:

- 1) Узел опитививает все роутеры (на $ff02::2^{102}$) сообщ. **ICMP RS** чтобы их найти в сети
- 2) Роутер в ответ опитр. все узлы (на $ff02::1^{103}$) сообщ. **ICMP RA**, чтобы получить с узлом адреса соседа

✓ Генерация хосингов ч. (Interface ID)

• Через интерфейс **EUI-64**

1. MAC-адрес делится на две равные части (по 24 бита в каждой), между которыми вставляются фиксированное число в шестнадцатеричной записи – FFFE (16 бит).
2. Происходит инвертирование седьмого бита в первом байте последовательности. Этот бит указывает на происхождение MAC-адреса (получен от производителя или настроен локально). Его инвертирование позволяет отличить MAC-адреса, выданные производителями сетевого оборудования, от назначенных сетевыми администраторами.



• Через криптографию

Хосингов число можно генерировать криптограф., чтобы было невозможным определить положение полей по MAC-адресу (или в EUI-64)

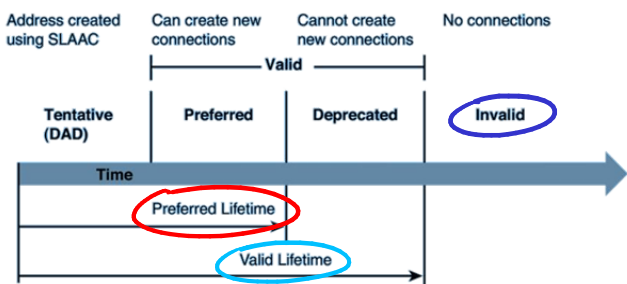
+ Статические
методы

▪ Статика + EUI-64/Криптография. В этом случае первые 64 бита задаются статически (вручную), а Interface ID генерируется с помощью EUI-64 или методов криптографии.

▪ Полностью статическая конфигурация адреса.

Время жизни SLAAC адреса

Адреса, сгенерированные по механизму SLAAC с псевдослучайным interface-ID, не используются бесконечно долго (в отличие от адресов, которые были назначены на интерфейс статически) и обладают несколькими периодами жизни (см. Рисунок 115).



~ часы / дни

Preferred lifetime

В течение этого интервала времени наше устройство все подключения устанавливает с этого адреса и все другие узлы, которые подключаются к нашему устройству, также при установлении соединения указывают этот адрес.

Valid lifetime

По истечении Preferred lifetime адрес еще какое-то время продолжает оставаться валидным (см. Рисунок 115), но в этот момент устройство должно сгенерировать новый адрес (в IPv6 на интерфейсе одновременно может быть несколько адресов из одной подсети, но находящихся в разных состояниях). Новый сгенерированный адрес находится в состоянии Preferred lifetime. Это означает, что все новые соединения, которые устанавливаются с нашим устройством и/или которые устанавливает наше устройство, будут использовать новый только что сгенерированный адрес. «Ста-

Invalid

По истечении Valid lifetime адрес переходит в состояние Invalid. Это означает, что с этого адреса уже нельзя ни устанавливать, ни принимать, ни поддерживать соединения (то есть текущие соединения, проассоциированные с этим адресом, будут разорваны).

NDP

— поиск MAC-адресов
рутеров и соседей

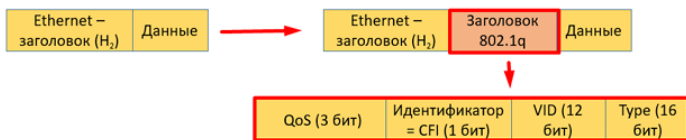
используя ICMPv6 (используя разные типы ICMPv6 пакетов), поиском соседей занимается протокол NDP, который в том числе позволяет выяснить MAC-адрес по известному IPv6 адресу. Задачи протокола NDP представляют собой поиск маршрутов в сети и поиск соседей. Поиск маршрутов определяется сообщениями RS и RA, о которых говорилось ранее, и необходим, например, для работы протокола SLAAC. Поиск соседей является аналогом работы протокола ARP в IPv4 и состоит из сообщений NS — Neighbor Solicitation (аналог ARP-запроса) и NA — Neighbor Advertisement (аналог ARP-ответа).

VLAN

VLAN (Virtual Local Area Network) — виртуальная

локальная компьютерная сеть. Представляет собой группу хостов с общим набором требований, которые взаимодействуют так, как если бы они были подключены к широкополосному домену независимо от их физического местонахождения¹⁰⁵.

Соединение trunk — такое соединение, по которому могут передаваться данные нескольких виртуальных сетей одновременно. Чтобы второй коммутатор, который получает от первого данные, мог понять, какой виртуальной сети эти данные принадлежат, **кадры, отправляемые в trunk, дополнительно маркируются**. Существует несколько протоколов, образующих trunk. Наиболее распространенный сегодня — IEEE 802.1q. Это протокол, который добавляет специальный заголовок после заголовка Ethernet (см. Рисунок 120).

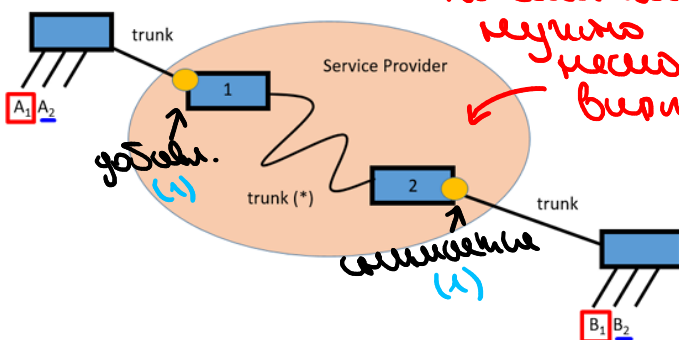


Итак, принадлежность к виртуальной сети может быть определена с помощью разных параметров: port based VLAN и tag based VLAN.

▪ **port based VLAN** — VLAN на основе физического порта: принадлежность к виртуальной сети в явном виде определяется по конкретному номеру интерфейса коммутатора (пример с делением физического коммутатора на логические).

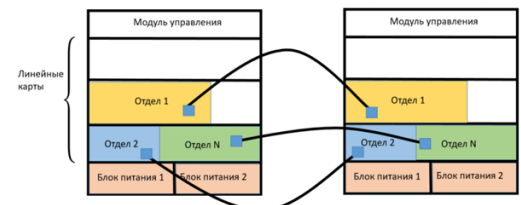
▪ **tag based VLAN** — VLAN на основе тега: принадлежность к виртуальной сети определяется дополнительной меткой в заголовке, по которой коммутатор понимает, к какой виртуальной сети принадлежит фрейм (пример с объединением виртуальных сетей двух физических коммутаторов с помощью trunk).

QinQ — технология, которая применяет два тега подряд: внешний тег и внутренний. Рассмотрим пример применения данной технологии. Допустим, мы обращаемся к провайдеру, чтобы он передал наши данные через свою сеть (объединил два наших офиса). То есть мы просим провайдера организовать некоторый L2 канал, как показано на схеме ниже (Рисунок 121) (в одном нашем офисе живут устройства A₁, в другом — B₁).

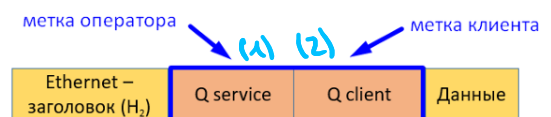


- CFI (Canonical Format Identifier) — указывает, с каким канальным протоколом используется этот заголовок: с Ethernet или с чем-то другим¹⁰⁶;
- VID (VLAN Identifier) — номер виртуальной сети;
- Type — то, что инкапсулировано далее (указатель на протокол третьего уровня = что инкапсулировано во фрейм).

Решение 1: каждый виртуальный коммутатор из первого устройства соединить физическим каналом с необходимым виртуальным коммутатором из второго устройства (см. Рисунок 118).



Решение 2: чтобы решить проблему на стороне оператора, описанную выше, была придумана технология QinQ — технология двойного тегирования, суть которой заключается в том, что у нас идет подряд две метки протокола 802.1q. То есть QinQ позволяет оператору добавить еще одну метку к уже существующей клиентской метке (см. Рисунок 122).



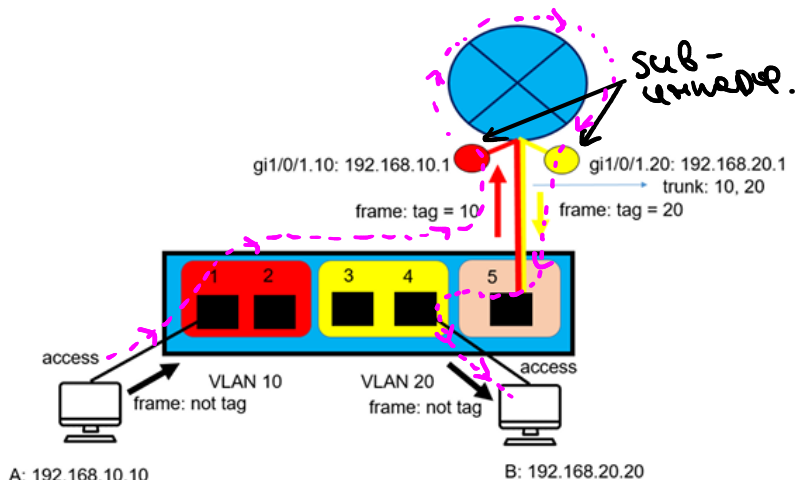
=> если провайдерский VLAN сопоставлен нескольким сетям клиента

А если клиент вводит свой VLAN 120! Но он знает.

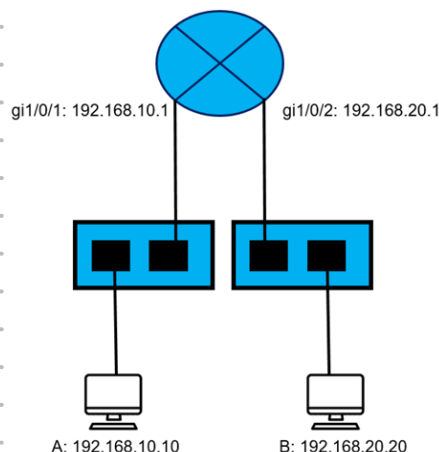
В этом случае некоторые операторы могут пойти навстречу клиенту и использовать технологию **VLAN Translation**. Эта технология позволяет заменить тег, с которым приходит трафик от клиента (например, от клиента приходит тег 120, а оператор заменяет его на 140).

Маршрутизация между VLAN: Router-on-a-stick

Хотим обсудить орг.
связь между разными VLAN



логическая схема



1) хост А понимает, что хост В расположен в другой подсети (прикладывает свою маску к адресу хоста В) и отправляет трафик на шлюз по умолчанию (предварительно с помощью ARP-запроса узнав MAC-адрес этого шлюза);

2) фрейм с хоста А доходит до первого порта коммутатора (пока нетегированный), коммутатор принимает фрейм, просматривает MAC-таблицу для VLAN 10 и отправляет данный фрейм через порт, который «смотрит» в сторону маршрутизатора (так как MAC-адрес для 192.168.10.1 он изучил оттуда), при непосредственной отправке через интерфейс коммутатор навешивает соответствующий данной виртуальной сети тег (10);

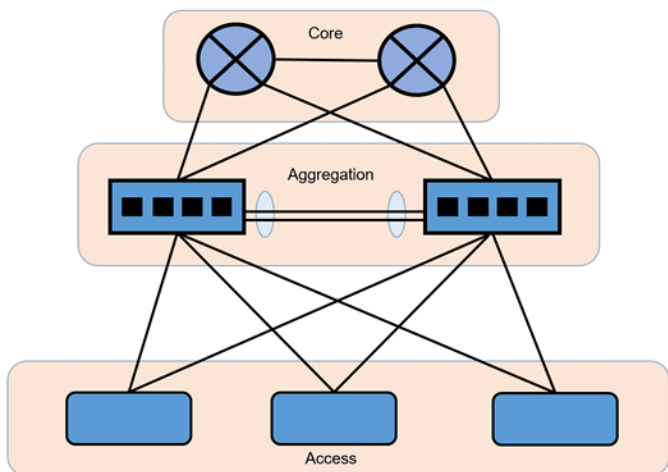
3) фрейм доходит до маршрутизатора и маршрутизатор ассоциирует его по тегу (10) с sub-интерфейсом gi1/0/1.10. Маршрутизатор просматривает свою таблицу маршрутизации и понимает, что адрес назначения находится за sub-интерфейсом gi1/0/1.20, маршрутизирует трафик в соответствующий sub-интерфейс (не забыв при этом изменить MAC-адреса отправителя и получателя, а также выполнить декремент поля TTL и пересчет контрольной суммы заголовка IP-пакета) и отправляет его через тот же физический интерфейс, но уже с тегом 20;

4) коммутатор, получив данный фрейм, просматривает MAC-таблицу VLAN 20 (так как фрейм пришел с тегом 20) и понимает, что получатель находится за четвертым портом. При отправке фрейма через четвертый порт тег снимается и нетегированный трафик отправляется на хост В.

Технология QinQ решает проблему нехватки виртуальных сетей у оператора, однако создает другую не менее опасную проблему – переполнение таблицы коммутации на коммутаторах провайдера. Это связано с тем, что оператору нужно изучить все MAC-адреса в своей сети: и собственные, и клиентские. Справиться с этой проблемой поможет другая технология – PVB¹⁰⁷. Логика работы PVB несколько отличается – клиентский фрейм в некотором смысле инкапсулируется внутри «родительского» кадра, таким образом в таблице коммутации устройств ядра чрезвычайно мало записей.

Порт коммутатора, который «смотрит» в сторону маршрутизатора, настраивается в режиме trunk и трафик 10 и 20 VLAN по нему передается тегированный. На маршрутизаторе появляется специальная конструкция, которая привязывается к физическому интерфейсу и называется sub-интерфейс. Каждый sub-интерфейс ассоциируется с конкретным VLAN (число привязываемых sub-интерфейсов ограничено числом VLAN). Таким образом, по номеру тега во входящем трафике маршрутизатор понимает, к какому sub-интерфейсу этот трафик относится, а весь исходящий с sub-интерфейса трафик тегуется номером VLAN, в соответствии которому поставлен данный sub-интерфейс.

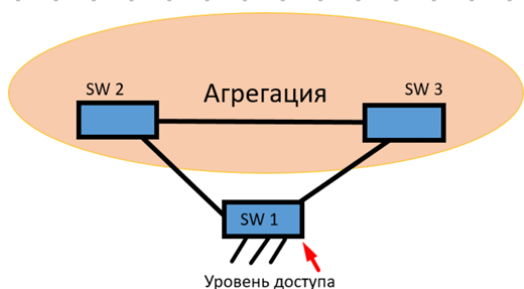
STP



Выделяют три уровня в архитектуре сети: ядро, агрегация и уровень доступа (см. Рисунок 127). В ядре, как правило, расположены L3 устройства (маршрутизаторы), на уровне доступа L2 устройства, то есть коммутаторы (отвечают за подключение пользователей), а на уровне агрегации происходит переход (то есть устройства и второго, и третьего уровней).

Однако проблемы возникают на уровне доступа: как только мы хотим задублировать какой-то канал, то у нас сразу возникает кольцо, по которому данные могут передаваться бесконечно долго.

В заголовке IP-пакетов есть поле, которое ограничивает количество переходов (время жизни пакета в сети) — TTL (Time To Live)¹¹⁰. На канальном уровне в Ethernet-кадре аналогов такого поля нет, то есть кадр может бегать по сети бесконечно долго.



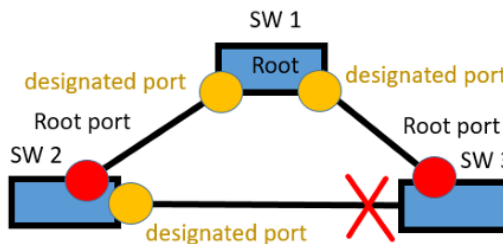
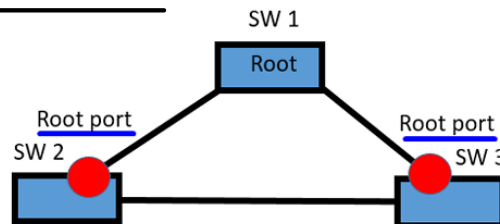
← нехитрое решение: STP

STP¹¹¹ — протокол, который позволяет разрывать петли (кольца/циклы) в L2 домене.

STP строит дерево с единственным корнем: такая архитектура исключает циклы.

Помощь робота STP:

1. Выбирается корень дерева (один из коммутаторов, которые есть в сети) — **Root switch**.
Допустим, в примере выше (Рисунок 129), корневым коммутатором будет Sw1.
2. Все остальные коммутаторы выбирают единственный **Root Port (RP)** — специальный интерфейс, путь от которого до корневого коммутатора наиболее короткий.
3. Для всех сетевых сегментов происходит выборы **Designated Port (DP)** — назначенного порта: тот интерфейс, через который трафик из этого сегмента может попасть к корню дерева наиболее коротким путём.
4. Все остальные интерфейсы, которые не являются root port или designated port должны быть заблокированы (виртуально, не физически) — пользовательский трафик через эти порты передаваться не будет (Рисунок 131).



Если в какой-то момент «падает» один из каналов (например, тот, который соединяет коммутаторы 1 и 3): протокол STP обнаруживает эту проблему и перестраивает дерево. В ситуации, когда «упал» канал между 1 и 3 коммута-

Выбор Root:

- 1) Избираются все сегменты сети Root и опис. BPDU-сообщ. об этом
- 2) Далее узлы выбирают узел с миним. BID:
BID¹¹³ = [priority (2 байта) + MAC (6 байт)]
- 3) Этот узел становится Root и регулярно рассылает BPDU-сообщ. о том, что он Root, остальные их не рассматривают

BPDU-сообщения, выбор портов

Root коммутатор через все свои интерфейсы начинает отправлять BPDU со следующей информацией:

- RID; ← **Root ID** не меняется при перес.
- свой BID (каждое устройство, которое будет пересылать это BPDU, будет в этом поле указывать своё значение BID);
- **Root path cost (RPC)** — стоимость пути до корневого коммутатора = насколько далеко от этого коммутатора до того, который выбран корневым.

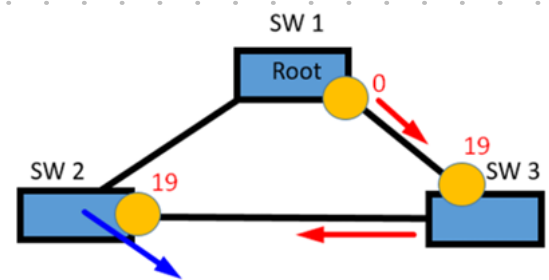
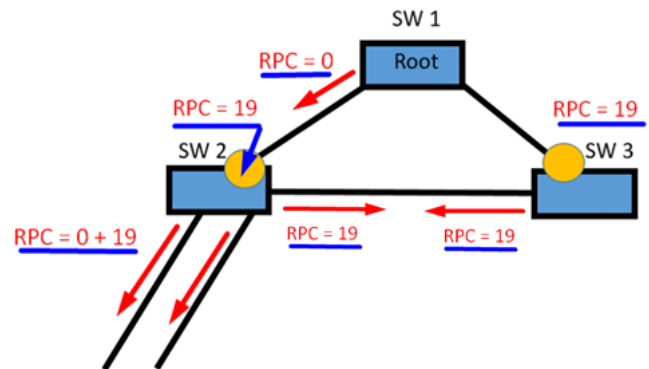
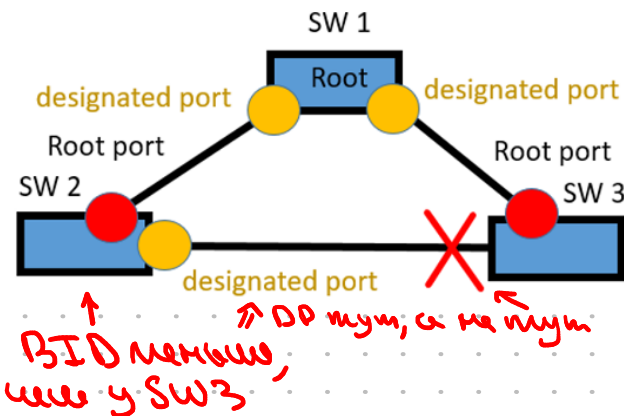
Коммутатор 1 сам является корневым и поэтому отправляет BPDU с Root path cost = 0

Коммутаторы, подключённые к Sw1, будут получать BPDU и будут добавлять к полученному Root Path Cost стоимость своего интерфейса, через который это объявление было получено. STP рассчитывает стоимость интерфейса на основании скорости, на которой этот интерфейс работает: чем выше скорость, тем меньше стоимость интерфейса. Предположим, что стоимость всех интерфейсов на нашей схеме равна 19.

Коммутатор SW 2 берёт RPC от соседа и прибавляет стоимость собственного интерфейса, через который это объявление было получено ($0 + 19 = 19$). То же самое делает Sw3 ($0 + 19 = 19$). Таким образом, когда дерево полностью построено и изменений в топологии нет, коммутаторы транслируют BPDU через все **Designated Port-ы** и в качестве Root Path Cost сообщают свое собственное значение Root Path Cost (см. Рисунок 134).

Определим, какие интерфейсы будут Designated Port. Назначенным портом будет тот, через который отправляется наименьшая стоимость пути до Root.

к нижнему каналу подключены два коммутатора, через которые до Root одинаковый путь. В этом случае происходит сравнение BID: коммутатор, у которого BID наименьший будет обслуживать этот сегмент.



стоимость пути через коммутатор 3 до Root = $19 + 19 + 0 = 38$

Рисунок 135. Логика изменения Root Path Cost при прохождении BPDU через коммутаторы.

Таким образом, коммутаторы посмотрят на интерфейсы, через которые доступен Root и выберут тот, через который стоимость пути до Root наименьшая. Именно этот интерфейс будет назначен Root Port (выберут, как на схеме выше (Рисунок 135)).

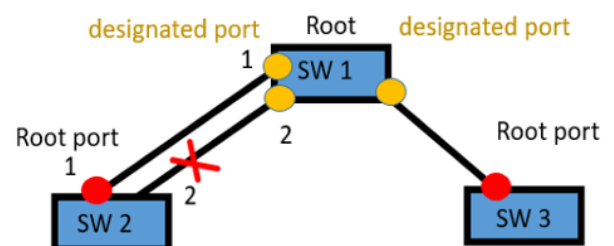
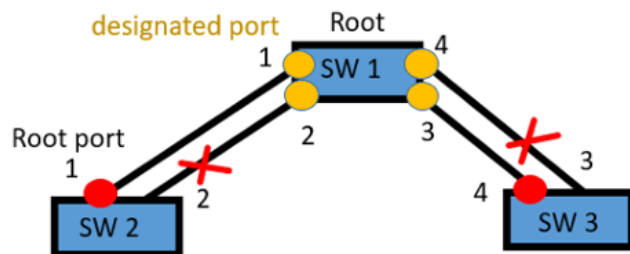
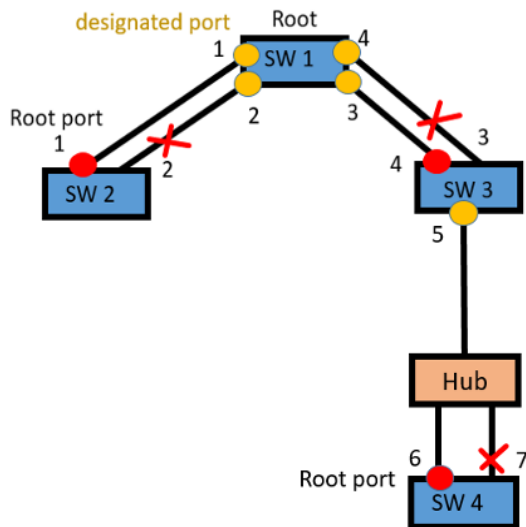


Рисунок 138. Ситуация, в которой выбор Root и Designated Port-а осуществляется по PID.

В этой ситуации мы смотрим на значение PID — идентификатор порта соседа. И коммутатор 2 выбирает тот канал, на котором **PID соседа наименьший** (на рисунке верхний канал). Отметим, что информация о PID интерфейсов коммутатора 1 также содержится в BPDU, которые от него прилетают.

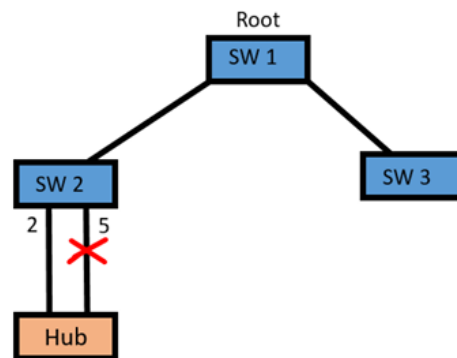


В этой ситуации мы также ориентируемся по номерам портов соседа. То есть коммутатор 3 выбирает тот канал, который подключён в меньший порт коммутатора 1

Рисунок 140. Ситуация, в которой выбор Root Port-а осуществляется по собственному PID.

Концентратор никак не участвует в STP. Через интерфейсы 6 и 7 коммутатора 4 приходят BPDU, отправленные коммутатором 3, в которых одинаковые RPC, BID и PID (=5). В этом случае коммутатор 4 будет ориентироваться на свой собственный PID и выберет среди своих портов наименьший (порт 6 – Root port, порт 7 – заблокирован).

аналогично



Таймеры в STP

- BPDU отправляе. root-ом каждые 2 сек
- После извлечения информации через 20 сек (10 BPDU-сообщ.)
- Спустя 20 сек роли коммутаторов начинают меняться

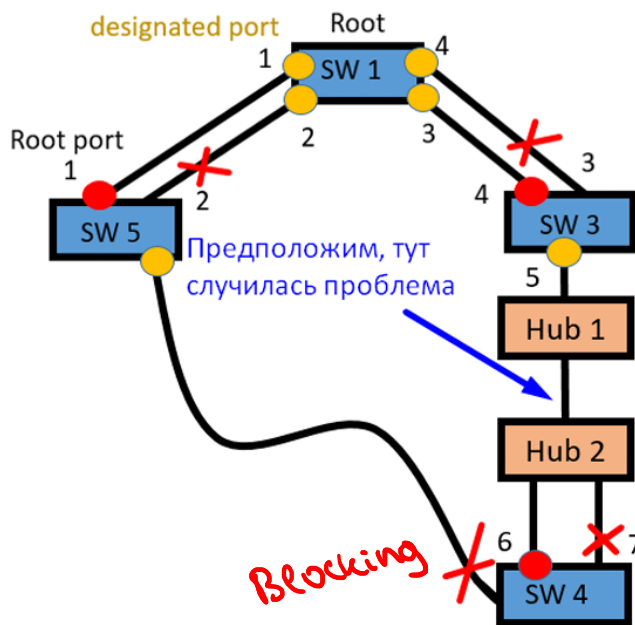
Что произойдет с интерфейсом SW3-SW4?

1. Изменяется от Down в сощм. **Blocking**

2. После 20-секундного ожидания BPDU-сообщ, от соседа переходим в сощм. **Listening** – 15 сек

3. Далее в сощм. **Learning** – 15 сек

4. Далее в сощм. **Forwarding** и тропы начинают обмениваться



* если порт самостоятельно извлечет своё состояние, то ему не нужно ждать 20 сек

Версии протокола STP

1) RSTP

В RSTP появился специальный механизм Proposal/Agreement, благодаря которому Rapid-STP сходится гораздо быстрее. Рассмотрим логику работы данного механизма (Рисунок 142).

1. Выбирается корень дерева (один из коммутаторов, которые есть в сети) – Root switch
2. Root switch (см. Рисунок 142 – это коммутатор 1) блокирует все интерфейсы и через них рассылает сообщение BPDU Proposal.
3. Получив такое сообщение, коммутаторы, которые находятся ниже (см. Рисунок 142 – это коммутаторы 2 и 3), блокируют все свои интерфейсы, кроме того интерфейса, через который был получен Proposal и отправляют «обратно» (через тот интерфейс, через который получили Proposal) сообщение Agreement.
4. Получив Agreement, от нижестоящих коммутаторов, корневой коммутатор разблокирует те интерфейсы, через которые был получен Agreement. Это означает, что в рассмотренной части сети петля точно нет.
5. Коммутаторы 2 и 3 поступают по примеру корневого коммутатора: рассылает через все заблокированные порты сообщение Proposal.

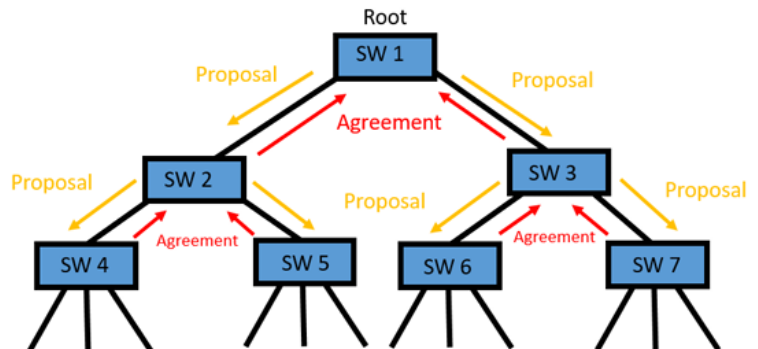
← *нормы, которые коммутаторы слышат RP*

← *нормы, которые коммутаторы слышат DP*

В протоколе RSTP изменился набор состояний и ролей портов. В частности, состояния Blocking и Listening были заменены одним состоянием – Discarding. В состоянии Discarding порт не пересылает и не обрабатывает полученные фреймы, не запоминает MAC-адреса, но при этом прослушивает входящие BPDU.

В RSTP остались такие роли портов, как корневой и назначенный, а роль заблокированного разделили на две новых роли: Alternate и Backup. Alternate — это резервный корневой порт, а Backup — резервный назначенный порт. Данные роли позволяют выполнить быстрое переключение в случае отказа.

Все универсально. В Full Duplex!



Сообщения Proposal и Agreement не полагаются на таймеры. Они отправляются, принимаются и обрабатываются настолько быстро, насколько это могут сделать процессоры коммутаторов. Именно это позволяет топологии сходиться гораздо быстрее, чем при классическом STP.

Состояния RSTP:

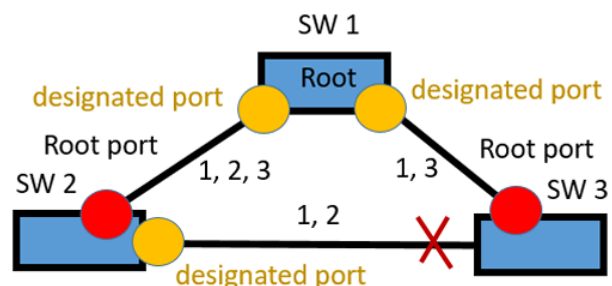
*Discarding
Learning
Forwarding*

Роли портов RSTP:

*Root Port (RP)
Designated Port (DP)
Alternate Port
Backup Port*

Проблемы RSTP:

Рассмотрим проблему, которую протокол RSTP решить не сумел: протоколы STP и RSTP не учитывают наличие VLAN-ов. Вне зависимости от того, есть в сети VLAN-ы или нет, данные протоколы строят одно единственное дерево.



VLAN-ы не работают...

2) PVST+ (Cisco)

PVST+ на каждый VLAN строит отдельное независимое дерево. То есть если в вашей сети три VLAN-а как в примере выше, то у вас будут три независимых дерева (а если 323 VLAN-а, то 323 независимых дерева). PVST+ решает проблему разорванных VLAN-ов: у каждого дерева может быть собственный корневой коммутатор.

Чтобы у каждого дерева мог быть выбран собственный корневой коммутатор, приоритет каждого коммутатора «квантуется»: значение приоритета можно указывать только с шагом 4096. Это связано с тем, что всего в сети потенциально может быть максимум 4096 VLAN-ов.

Приоритет коммутатора для конкретного VLAN = Приоритет для коммутатора + Номер VLAN¹¹⁹.

3) Rapid PVST+ (Cisco)

Rapid PVST+ совмещает в себе идеи протоколов PVST+ и RSTP, используя RSTP для построения каждого отдельного дерева. Другими словами, Rapid PVST+ запускает отдельный экземпляр RSTP для каждой виртуальной локальной сети (VLAN).

4) VBST (Huawei)

Полностью аналогично Rapid PVST+

5) MST/MSTP

MSTP в отличие от протокола Rapid PVST+ учитывает их с использованием некоторой агрегации – групп VLAN-ов (такие группы называются **instance**). Администратор вручную добавляет в каждый instance набор VLAN-ов и уже после этого для каждого instance MSTP строит независимое дерево. То есть количество деревьев будет значительно меньше, чем количество VLAN-ов в сети. Такой подход оправдан, так как в действительности количество различных и полностью независимых физических топологий сильно меньше, чем количество VLAN-ов в сети.

Оптимизации STP

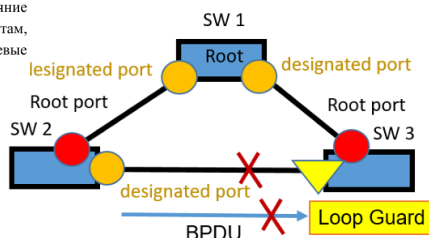
1) Root Guard

Root Guard можно включить на всех портах коммутатора, которые не являются корневыми (и не могут при авариях стать корневыми). Обычно эта функция включена на портах, подключенных к граничным коммутаторам, которые никогда не должны получать пакеты BPDU с лучшим BID. Каждый коммутатор должен иметь только один корневой порт, который является наилучшим путем к корневому коммутатору.

Root guard гарантирует, что порт, на котором включена данная оптимизация, является назначенным портом. Если мост получает superior BPDU на порту с активированной функцией Root Guard, то тогда Root Guard переводит этот порт в особое состояние ошибки. Это несогласованное с корнем состояние¹²² фактически равно состоянию прослушивания¹²³.

Решение:

Loop Guard. Когда коммутатор отправляет трафик, но неожиданно перестает принимать BPDU на таком интерфейсе, опция Loop Guard переводит интерфейс в состояние Loop Inconsistent и блокирует весь пользовательский трафик.



2) Loop Guard

Проблема:

Если в сети с одним корневым коммутатором произойдет обрыв одного из волокон, это приведет к появлению альтернативного пути к корню.

Допустим, произошёл обрыв одного из волокон между коммутаторами SW2 и SW3. В результате SW3 не получает никаких BPDU от SW2, но он всё ещё может отправлять трафик на SW2.

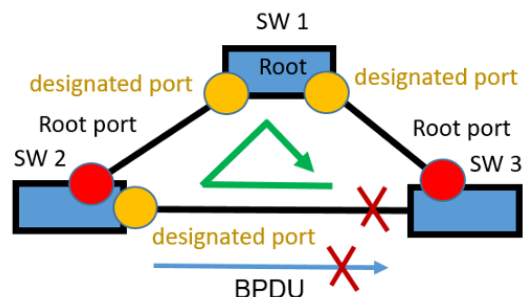


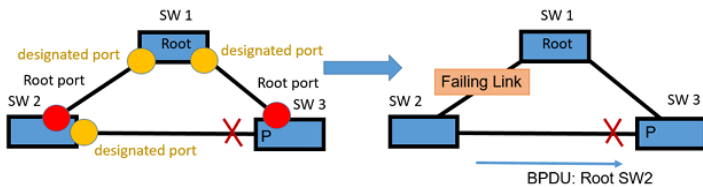
Рисунок 147. Пример аварии на физическом линке.

Поскольку SW3 больше не принимает BPDU на свой альтернативный порт, он перейдет в режим пересылки (forwarding). Теперь у нас есть односторонний цикл, как указано зеленой стрелкой.

3) Port Fast — чтобы не ждать 30 сек

PortFast — функция, которая позволяет порту пропустить состояния listening и learning и сразу же перейти в состояние forwarding. Настраивается на портах уровня доступа, к которым подключено пользовательское оборудование: компьютеры, сервера, принтеры, маршрутизаторы, брандмауэры.

5) Backbone Fast



Чтобы не ждать 50 сек:

Коммутатор SW3 получает сообщения inferior BPDU через свой заблокированный порт, в которых указано иное значение RID, чем у текущего корневого моста. Коммутатор отправляет RLQ¹²⁵ через все свои порты, являющиеся не назначенными, то есть через корневой порт и заблокированные порты. Это сообщение должно дойти до корневого коммутатора и вернуться обратно. Этим проверяется живость текущего корневого коммутатора. Когда RLQ возвращается обратно, SW3 разблокирует свой порт P и начинает через него отправлять BPDU, чем значительно экономит время перевода порта в режим forwarding.

4) BPDU Guard

BPDU Guard предотвращает вредоносные атаки и ошибки коммутации на пограничных портах и должна быть активирована на тех интерфейсах, которые не должны получать BPDU от подключенных устройств. К таким портам, например, относятся интерфейсы коммутатора в сторону пользовательского оборудования.

6) Bridge Assurance

На межкоммутаторных линиях при использовании протокола RSTP или более старших версий (канал должен работать в режиме point-to-point) включается отправка и анализ принимаемых BPDU на портах, находящихся в статусах Alternate и Blocking. Как только на порт перестают приходить BPDU, либо начинают приходить BPDU, в которых роль/состояние порта указывается некорректным, то порт блокируется, чем отсекаются сценарии возникновения петель из-за однонаправленных каналов, и временных петель из-за перестроения сети.

Технологии агрегирования каналов

Хотим соединить компьютеры или серверы, поставив вышесказанное в виде моста. Но вышесказанное объединяет STP из-за возник. коллизии.



решение: Технологии агрег. каналов

LAG:

Объединение нескольких физ. каналов в один логический



Объединить можно каналы, у которых совпадают следующие параметры конфигурации:

- скорость,
- режим дуплекса,
- native VLAN (тот VLAN, трафик которого передается без тегов¹²⁶),
- диапазон разрешенных VLAN,
- trunking status (все интерфейсы, которые объединяются, должны находиться либо в режиме access, либо в режиме trunk),
- тип интерфейса (под витую пару/оптическое волокно).

При этом STP не увидит, что там два физ. канала

Способы построения LAG

✓ **Статическое объединение каналов.** В этом случае вы в явном виде указываете интерфейсы, которые входят в Port Channel. При этом никакого согласования конфигурации и параметров с противоположной стороной не происходит.

✓ **Динамическое объединение каналов с использованием специальных протоколов (LACP/PAGP).** Чаще всего используется протокол LACP (Link Aggregation Control Protocol), при этом у компании Cisco есть собственный проприетарный протокол PAGP (Port Aggregation Protocol), но практически не используется. В случае динамического объединения каналов мы также должны указать интерфейсы, которые будем объединять в Port Channel. После этого на них поднимается протокол LACP и этот протокол должен согласовать Port Channel с устройством, которое подключено с другой стороны.

Более надежно, но имеет задержку

Если по какой-то причине согласование не произошло, то порты соответственно не объединяются в группу и не формируют PortChannel. В этой ситуации возможно несколько сценариев работы:

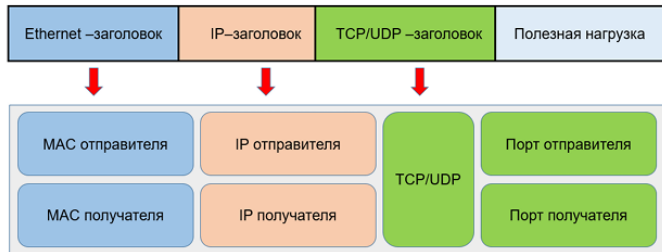
- каждый порт работает независимо;
- один порт работает независимо, а остальные переходят в состояние down;
- все порты не функционируют и ждут того момента, когда удастся согласовать Port-Channel с противоположной стороной.

Протокол LACP позволяет одновременно объединить до 16 интерфейсов в Port Channel. При этом передавать пользовательский трафик в определённый момент времени могут не более 8 интерфейсов. Еще 8 интерфейсов будут находиться в «горячем резерве»: при отказе одного из работающих интерфейсов на его место встанет один из резервных.

Балансировка трафика в LAG

Устройства, которые принимают решения о передаче трафика в Port Channel, могут учитывать ряд параметров, на основании которых вычисляют значение хэш-функции.

хэш в нашем случае можно представить следующей функцией (см. Рисунок 155): $\text{Hash} = f(\text{MAC}_s, \text{MAC}_d, \text{IP}_s, \text{IP}_d, \text{Protocol}(\text{TCP/UDP}), \text{Port}_s, \text{Port}_d)$.



$$\text{Hash} = f(\text{MAC}_s, \text{MAC}_d, \text{IP}_s, \text{IP}_d, \text{Protocol}(\text{TCP/UDP}), \text{Port}_s, \text{Port}_d)$$

В простейшем случае выбор пути определяется остатком от деления хэша на число доступных путей.

Технология EtherChannel использует похожий подход, однако предлагает брать остатки от деления не на число доступных путей (количества интерфейсов в агрегированном канале), а на максимальное число путей, которое поддерживает LACP (то есть 8). И дальше, в зависимости от количества физических каналов в Port Channel, в соответствие каждому пути ставить один (если каналов 8) или большее количество остатков (см. Таблица 24).¹²⁹

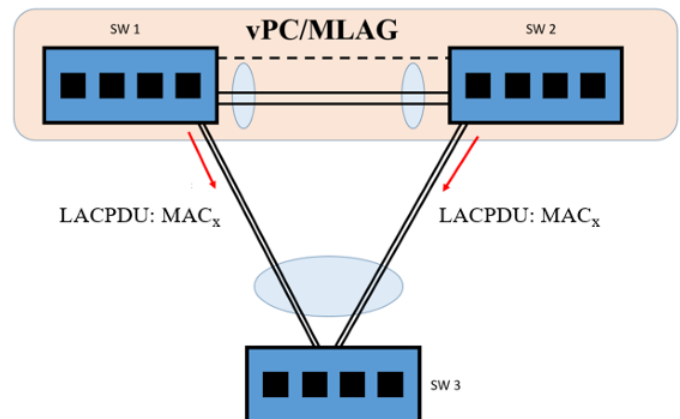
Таблица 24. Распределение остатков от деления хэша по интерфейсам.

Количество каналов в EtherChannel	Распределение потоков по каналам
2	4:4
3	3:3:2
4	2:2:2:2
5	2:2:2:1:1
6	2:2:1:1:1:1
7	2:1:1:1:1:1:1
8	1:1:1:1:1:1:1:1

Оптимизация LAG

1) vPC/MLAG/VSS

в решениях для ЦОД и крупных кампусных сетей вы можете встретить конструкции, которые называются vPC (virtual PortChannel)/MLAG (Multi-chassis LAG)/VSS (Virtual Switch System) и им подобные, которые обычно используется на уровне агрегации/дистрибуции. В таком решении два коммутатора объединяются в vPC/MLAG-пару и представляются для всех остальных устройств как единое целое (в том числе отправляют одинаковый MAC-адрес устройства при согласовании LACP). То есть вы можете построить на коммутаторе SW3 (см. Рисунок 159) каналы и до SW1, и до SW2 и всю эту конструкцию объединить в один Port Channel.



OSPF

OSPF¹³¹ – протокол динамической маршрутизации, который относится к классу гибридных протоколов. Это связано с тем, что OSPF сочетает в себе свойства и Link State, и Distance Vector протоколов. В OSPF вводится новый для нас термин – зона.

Зона (area) – некоторая совокупность маршрутизаторов (а точнее их интерфейсов).

Таким образом, OSPF-домен делится на части, каждая такая часть называется зоной. Разделение OSPF-домена на зоны проходит по интерфейсам маршрутизатора – зоне принадлежит не роутер целиком, а какая-то часть его портов

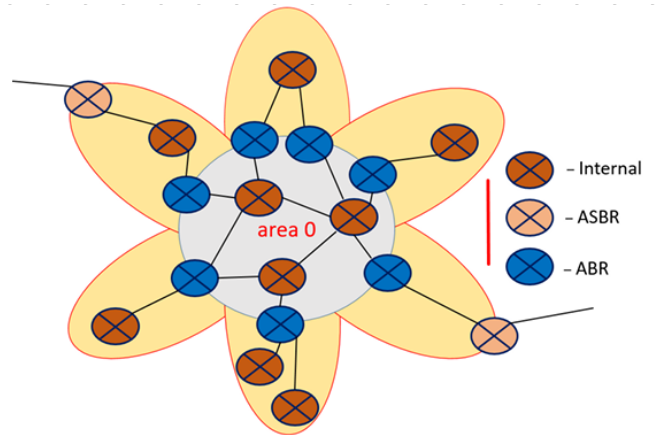
OSPF ведёт себя как LS-протокол внутри зон, и как DV-протокол между зонами, т.е. из других зон переключенные маршруты передаются до комп. узлов

Типы зон

- **area 0 (backbone)** – магистральная зона, к которой подключаются все остальные зоны. На схеме ниже (Рисунок две зоны, которые не являются магистральными, не могут быть соединены без подключения к area 0¹³².
- **area k (standard)** – стандартная зона с номером k , $k \geq 1$.
- **Stub (тупиковая зона)** – зона, в которую не могут попасть внешние относительно нашего OSPF-домена маршруты.
- **Totally Stubby (полностью тупиковая зона)** – зона, в которую не могут попасть не только внешние маршруты, но и маршруты из других зон.

Типы роутеров

- **Internal** (внутренний роутер) – маршрутизатор, все интерфейсы которого принадлежат одной зоне. При этом это может быть любая зона, не обязательно area 0.
- **ABR¹³³** (пограничный роутер) – роутер, часть интерфейсов которого расположена в нулевой зоне (хотя бы один), а часть интерфейсов расположены в какой-то другой (или других) зонах (хотя бы один). Условие нахождения
- **ASBR¹³⁵** (пограничный роутер системы/домена) – роутер, который стоит на границе между нашим OSPF-доменом и какими-то другими доменами (RIP, EIGRP, BGP, другой OSPF-домен).



Типы маршрутов

Рассмотрим более чётко маршруты, которые определяются в OSPF (мы их уже так или иначе определяли, вводя различные зоны).

- **Intra-zone** (внутризоновые) – маршруты для префиксов, принадлежащие одной зоне;
- **Inter-zone** (межзональные) – маршруты для префиксов, которые принадлежат нашему OSPF-домену, но расположены в других зонах;
- **External** (внешние) – маршруты, которые попали в OSPF из какого-то другого протокола (например, RIP или статика) = внешнего источника относительно нашего OSPF-домена.

Tunn LSA

LSA1 (Router LSA) – описывает маршрутизатор и его интерфейсы. Другими словами, роутер описывает сам себя: куда подключены его интерфейсы, какие соседства установлены. Собрав информацию из LSA1 со всех роутеров в рамках зоны, можно понять, как соединены все роутеры этой зоны между собой.

LSA2 (Network LSA) – позволяет построить топологию в случае, если в зоне есть коммутатор (или хаб, если в сети используется очень старое оборудование), к которому подключены роутеры.

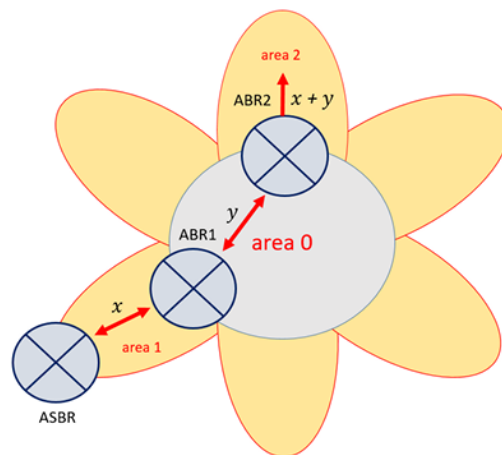
LSA1 и LSA2 достаточно, чтобы построить топологию внутри одной зоны.

LSA3 (Summary LSA) – передаёт информацию о маршрутах из другой зоны. В LSA3 содержится префикс и метрика – расстояние от ABR до этого префикса. При этом с помощью LSA3 нельзя построить топологию.

LSA5 (Autonomous system external) – предназначена для передачи внешней маршрутной информации внутри OSPF-домена. LSA5 также, как и LSA3, проходит между зо-

LSA4 (Summary ASBR) – позволяет посчитать расстояние от ABR до ASBR. То есть ABR генерирует LSA4 в нулевую зону, в которой сообщает, что знает маршрут до конкретного ASBR и значение метрики до этого маршрута (см. Рисунок 163). Совокупность информации в LSA4 и LSA5 позволяет отправлять трафик во внешние сети.

Рассмотрим ситуацию, когда информацию во внешнюю сеть нужно отправить не из зоны 0, а из какой-то другой зоны (см. Рисунок 163). В этой ситуации ABR2 сгенерирует LSA4, в которой укажет расстояние до ASBR другой зоны = *расстояние (ASBR, ABR1) + расстояние (ABR1, ABR2)*. Первое слагаемое ABR2 получит из LSA4, которое сгенерировал ABR1, второе слагаемое ABR2 известно, потому что у него есть вся топологическая информация о магистральной зоне.



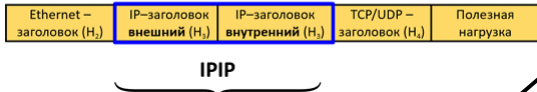
Метрика в OSPF

Определим, как посчитать метрику в OSPF. Общая метрика до сети получателя состоит из суммы метрик (стоимостей) каждого канала по пути. Стоимость каждого канала по умолчанию вычисляется по формуле: *стоимость канала* = $(Ref. BW / BW)$. В знаменателе реальная полоса пропускания канала, а в числителе фиксированное значение (по умолчанию 100 Мбит/с).

Туннели

В рамках темы Туннели мы рассмотрим ситуации, когда порядок инкапсуляции меняется, а некоторые инкапсуляции выполняются несколько раз на одном уровне. В частности, IP-пакеты вкладываются

Туннели IPIP. Рассмотрим самый простой пример туннелей, построенных на базе технологии IPIP. Эта технология представляет собой вложение одного IPv4 пакета внутри другого IPv4 пакета. Рассмотрим последовательность заголовков для туннелей IPIP (см. Рисунок 165).



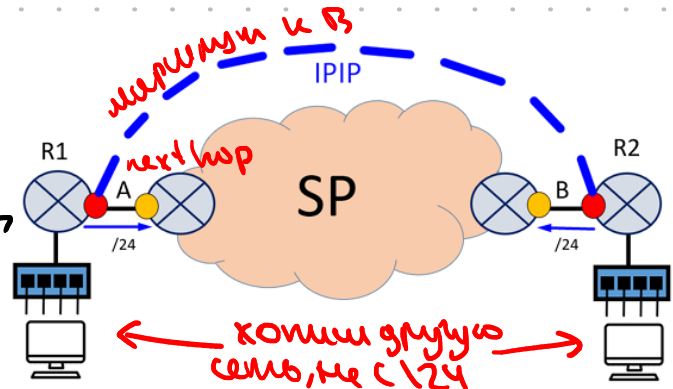
вирт.помещ.

Создаем туннельный интерфейс. R1 при передаче через него, помещает IP-пакет в другой IP-пакет

В частности, внутри внешнего IP-пакета можно инкапсулировать только другие IPv4-пакеты. Если, например, клиентские сети построены на IPv6, то необходимо построить уже другой туннель – IPv6IP

Лучше так:

GRE туннели. В этом решении между внутренним и внешним пакетами вставляется специальный GRE заголовок с указанием на то, что инкапсулировано внутри. То есть достаточно построить один туннель, через который можно передавать и IPv4, и IPv6, и ряд других протоколов, которые используют клиенты (см. Рисунок 169). К минусам этой технологии можно отнести дополнительные четыре байта, которые составляют GRE-заголовок (соответственно, полезная нагрузка будет на четыре байта меньше).



На этом этапе между роутерами R1 и R2 поднимается туннель. Для туннеля вводится понятие **endpoint**: точки, между которыми поднимается туннель. То есть на каждом из роутеров (R1 и R2) необходимо указать локальную и удаленную «точки приземления» туннеля (**local endpoint, remote endpoint**). В данном примере туннель поднимается между адресами R1 и R2, которые смотрят в сеть провайдера (соответствующие интерфейсы выделены красным). Именно эти адреса будут записаны во внешнем IP-заголовке пакета

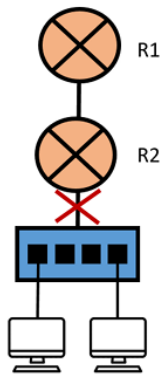
FHRP

FHRP (First Hop Redundancy Protocol) – это название семейства протоколов, которое отвечает за резервирование шлюза по умолчанию (default gateway) и повышает отказоустойчивость сети. Другими словами, эта группа протоколов позволяет избежать больших потерь трафика в случае, если на шлюзе по умолчанию возникли проблемы.

→ Производим
HSRP
GLBP
VRRP ← *исп.*

Работы HSRP

Рассмотрим, ситуацию, когда на R2 произошла какая-то проблема: это может быть как полный выход из строя данного маршрутизатора, так и выход из строя какого-либо из его интерфейсов (в нашем случае, например, в сторону коммутатора доступа). При возникновении такой проблемы (см. Рисунок 172) все сети доступа, которые терминировались на R2, будут недоступны (отрезаны от ядра сети).



=> *нужно резервировать R2*

На интерфейсах маршрутизаторов, которые смотрят в сторону пользователей (то есть по сути могут являться для них шлюзом по умолчанию) настраивается HSRP-группа. После этого маршрутизаторы обнаруживают друг друга и устанавливают отношения соседства по протоколу HSRP (см. Рисунок 174). В результате одно устройство выбирается активным (роль Active), а второе резервным (роль Standby).

при конфигурации протокола HSRP вы указываете виртуальный IP-адрес (VIP). Именно активный маршрутизатор обрабатывает трафик, который отправляется на виртуальный IP-адрес (VIP) и виртуальный MAC-адрес (vMAC). По сути пара VIP и vMAC расположены на активном маршрутизаторе в дополнение к стандартному MAC-адресу и IP-адресу, который вы настроили на интерфейсе. В случае отказа активного маршрутизатора конструкция {VIP+vMAC} переедет на Standby маршрутизатор, и он станет активным.

Как работает VIP и vMAC?

Клиент по протоколу ARP пытается выяснить MAC-адрес для VIP-адреса: широковещательный ARP-запрос получают все устройства, но отвечает на этот запрос только маршрутизатор с ролью Active и отправляет не MAC-адрес своего физического интерфейса, а vMAC. Таким образом у клиента в ARP-таблице формируется запись, которая связывает VIP и vMAC.

Если с Active маршрутизатором что-то случилось и VIP+vMAC переехали на R3 (см. Рисунок 175), то R3 берет на себя роль Active и отправляет GARP¹⁴², чтобы переобучить коммутаторы.

* HSRP *исп. можно* где угодно. и одну интерфейсы

* HSRP *масштабируемые* где одного VLAN независимо

Механизм Preempt:

Если он выше чем он роутирует, то мы можем. и если не выше чем роутирует Active-роутирует. Вызываем роутирует с миним приорит. (или с миним. IP-адресом инициируется HSRP)

VRRP

- Вместо ролей Active/Standby введены роли Master/Backup;
- Preempt включен по умолчанию;
- В VRRP по умолчанию hello¹⁴⁵ таймер равен 1 секунде, dead¹⁴⁶ таймер равен 3с (у HSRP 3с и 10с соответственно)
- Виртуальный адрес может совпадать с адресом интерфейса (то есть можно использовать только два адреса для конфигурации данного решения), хотя обычно такое решение не рекомендуется.